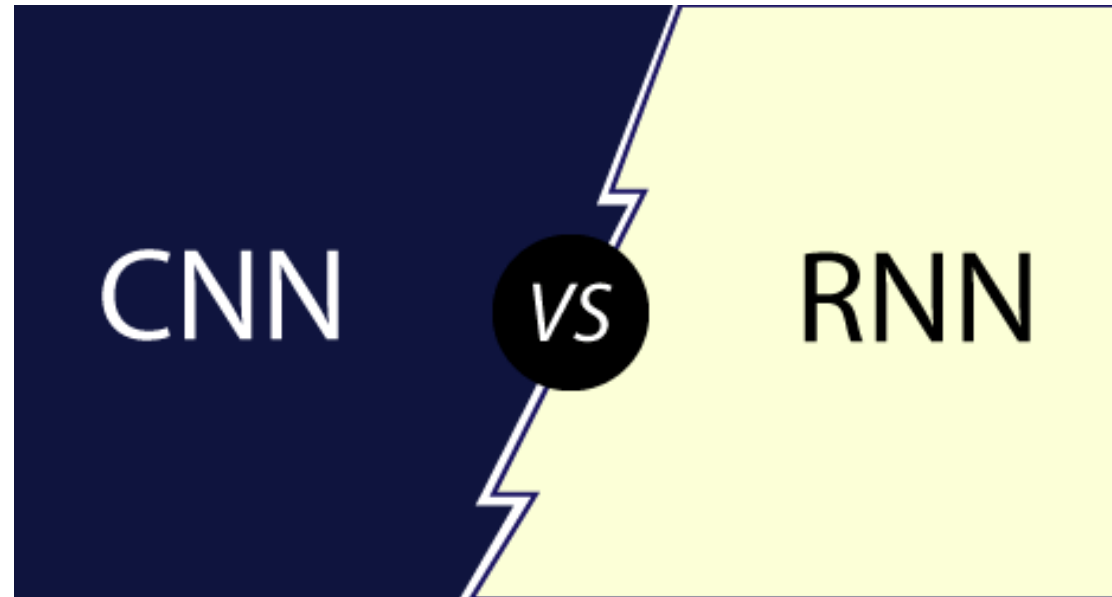


AIAA 2205: Introduction to AI Recurrent Neural Network/Transformer

Li LIU

avrillliu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)



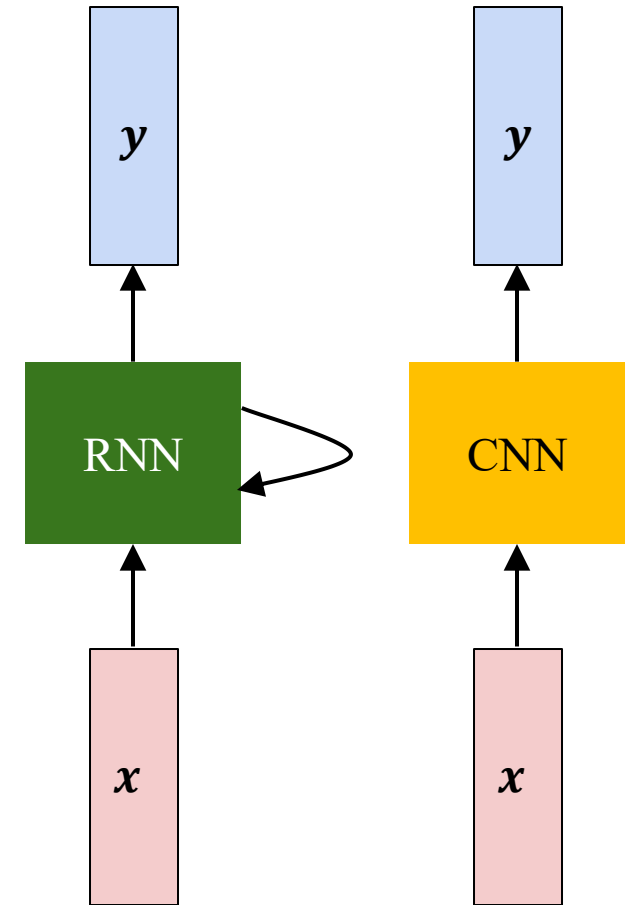
CNNs can “**understand**” images,
RNNs can “**remember**” the order of the sequence data.

Recurrent Neural Network

We can process a sequence of vectors $x = \{x_1, \dots, x_T\}$ by applying a recurrence formula at every time step t :

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state some function with parameters W old state input vector at some time step



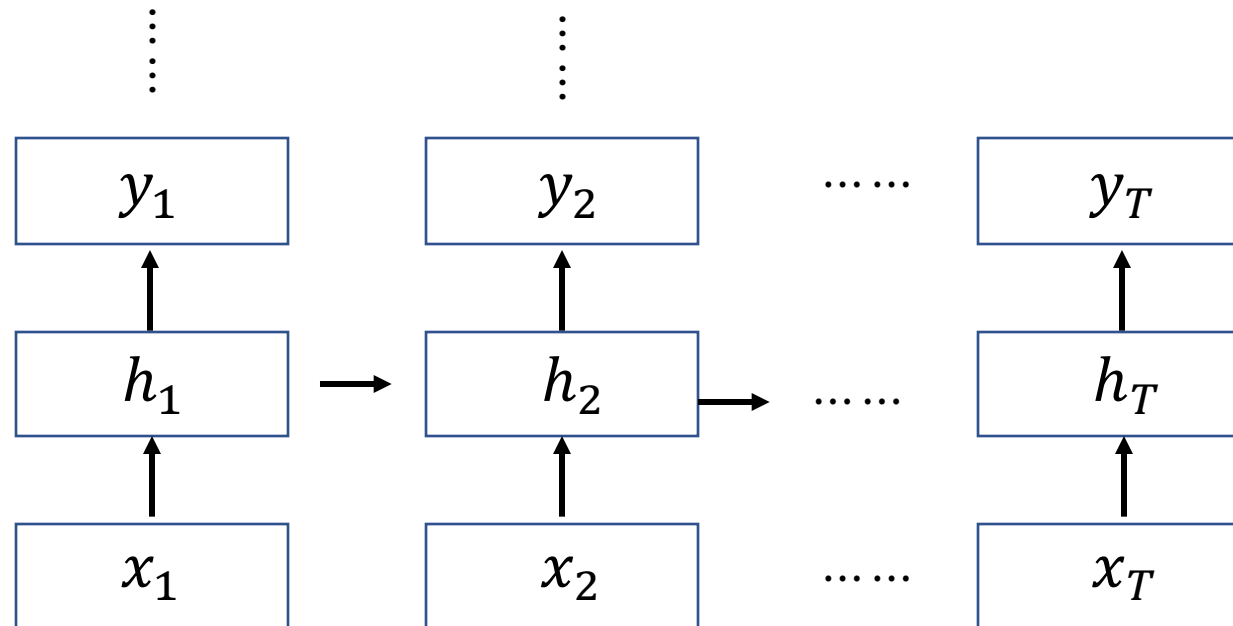
Fold RNN

CNN

One has the curve with arrow

Standard unrolled RNN

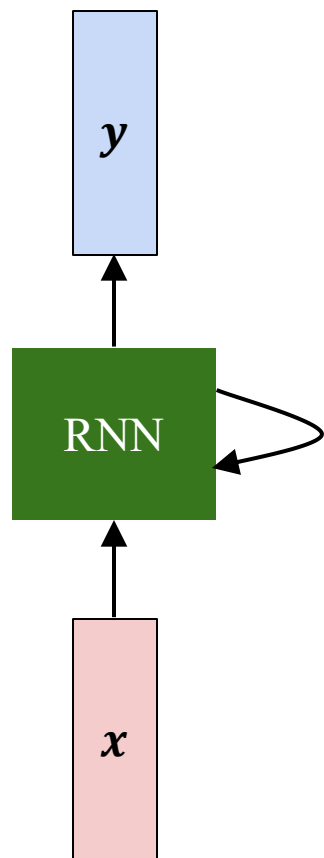
Deep in terms of **layers**



Deep in terms of **time steps**

(Vanilla) Recurrent Neural Network

The state consists of a single “*hidden*” vector h :



$$h_t = f_W(h_{t-1}, x_t)$$

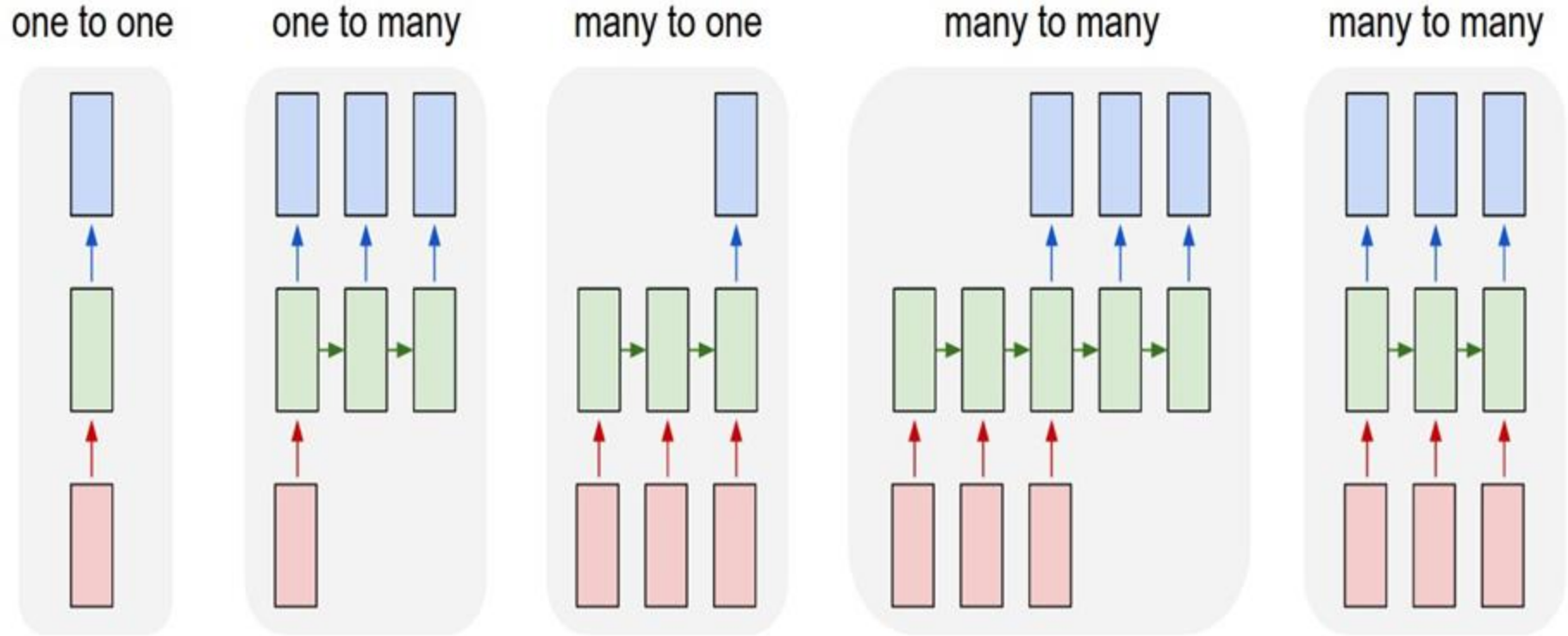


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

here neglect the bias b

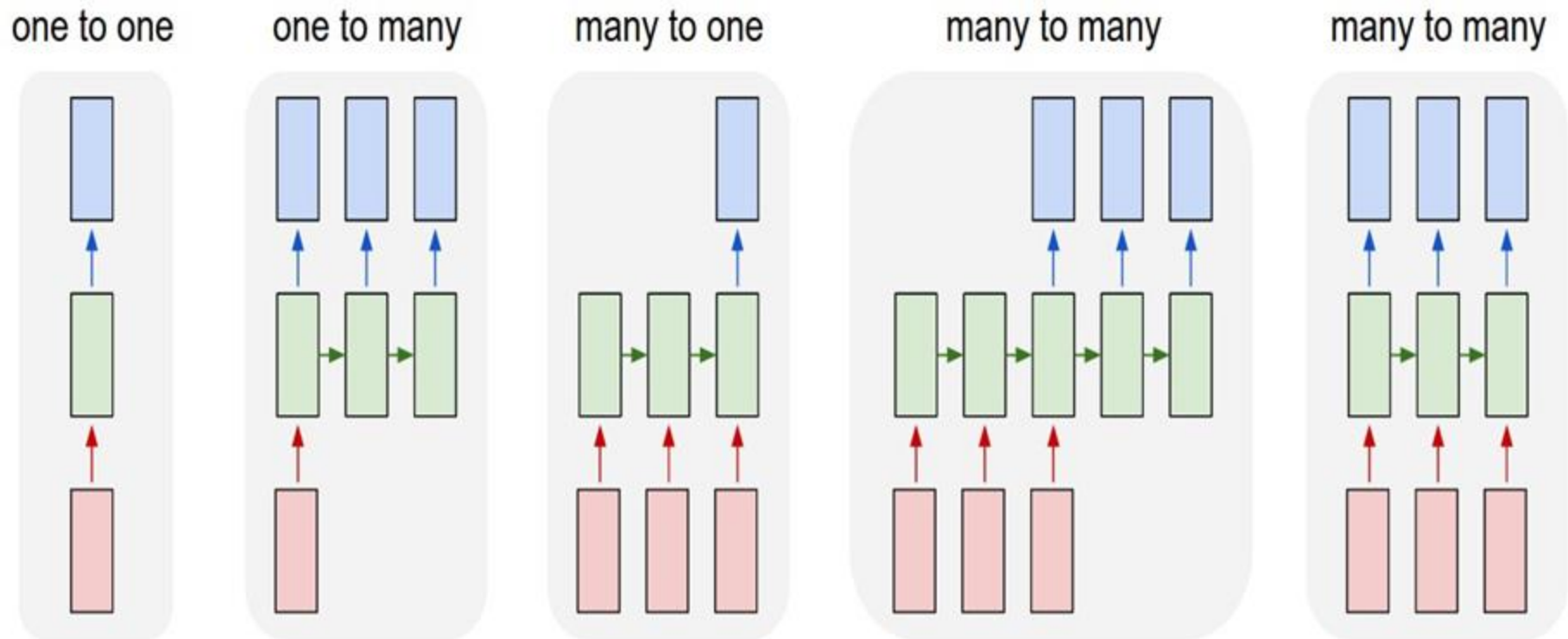
$$y_t = W_{hy}h_t$$

Recurrent Networks offer a lot of flexibility



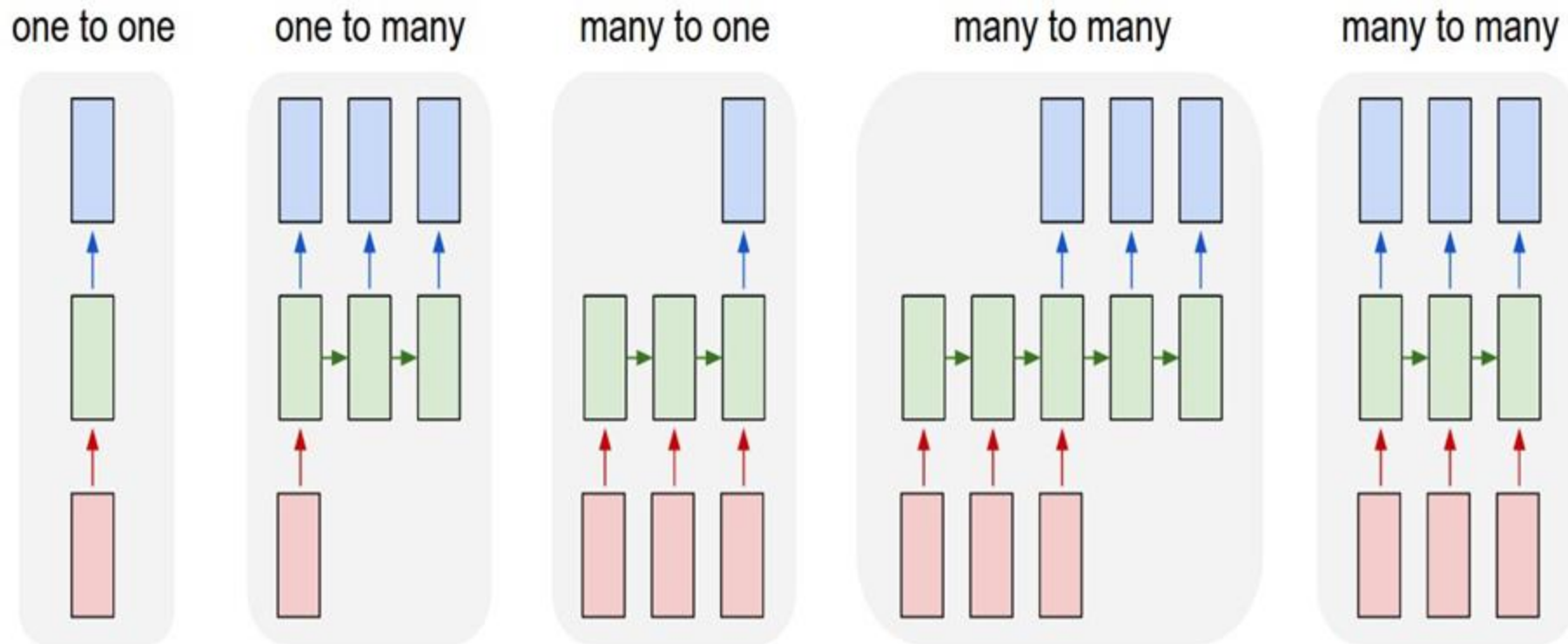
Vanilla Neural Networks

Recurrent Networks offer a lot of flexibility



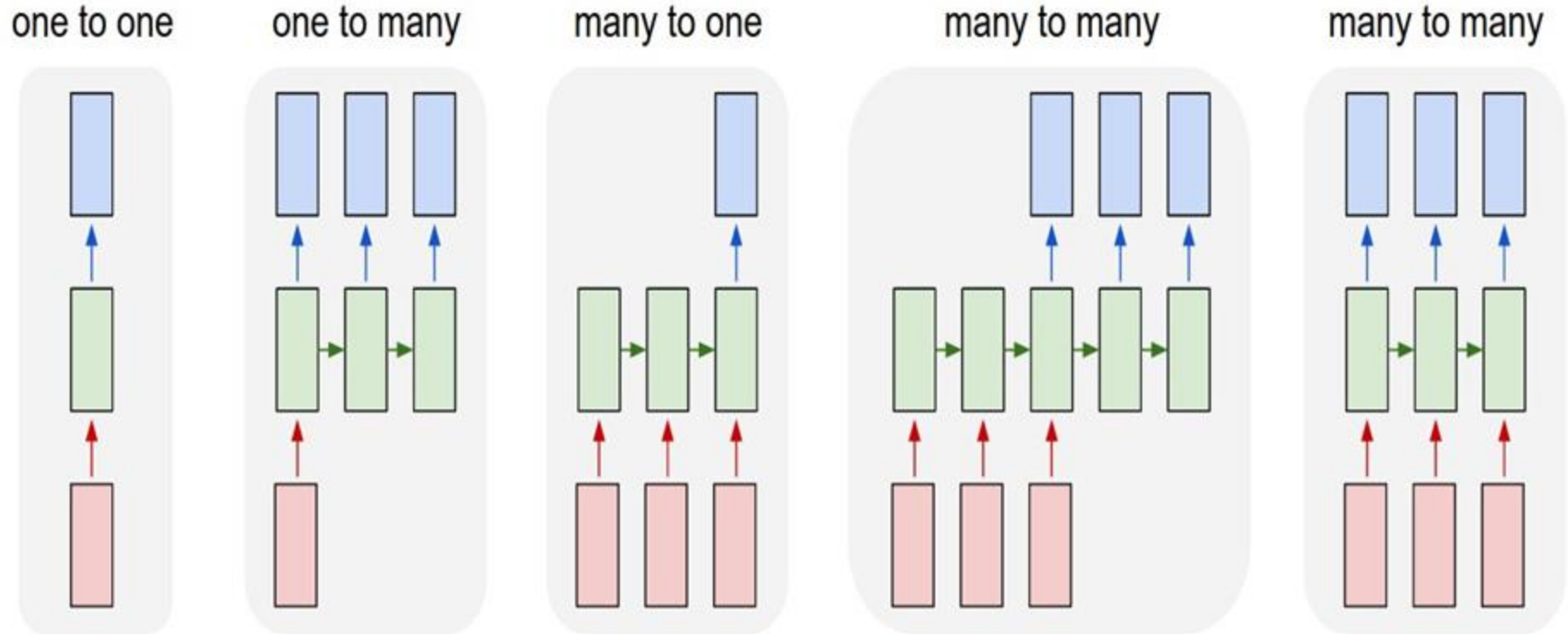
e.g., **Image Captioning**
image -> sequence of words

Recurrent Networks offer a lot of flexibility



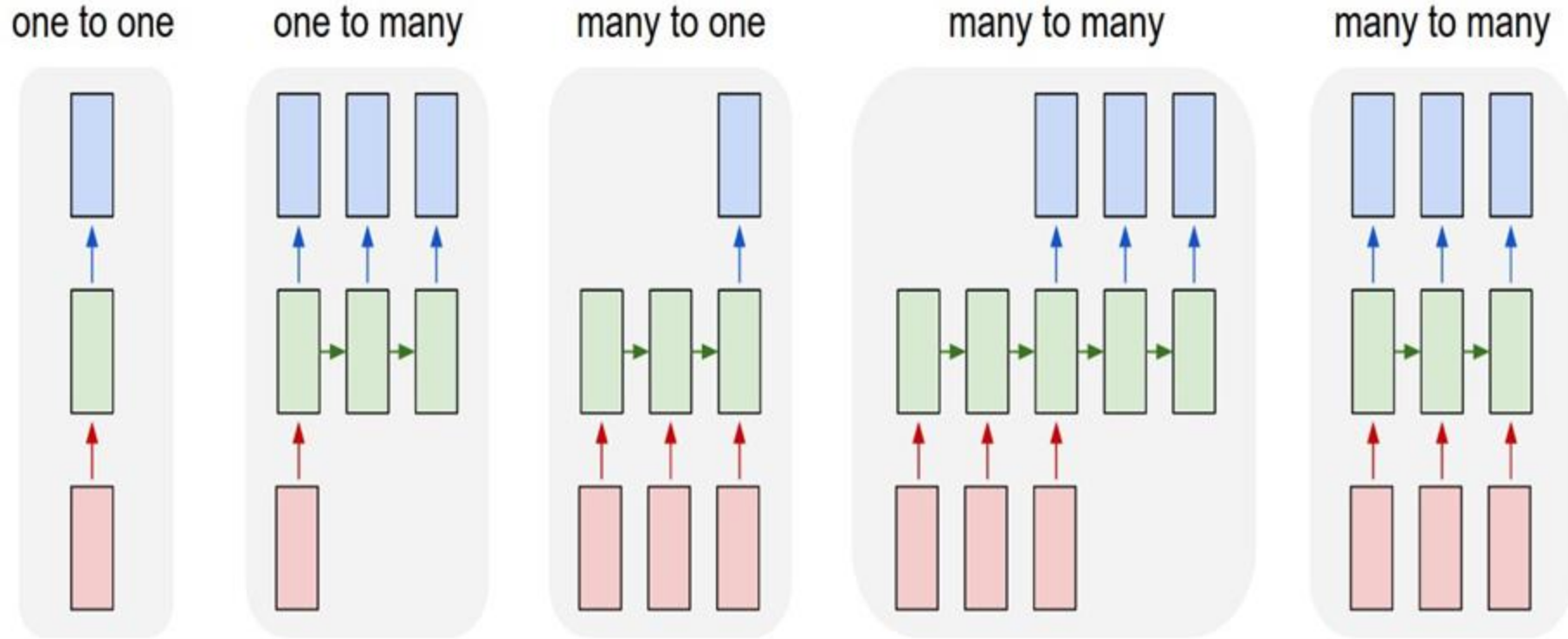
e.g., Sentiment Classification
sequence of words $\rightarrow$ sentiment

Recurrent Networks offer a lot of flexibility



↖
e.g., **Machine Translation**
seq of words → seq of words

Recurrent Networks offer a lot of flexibility



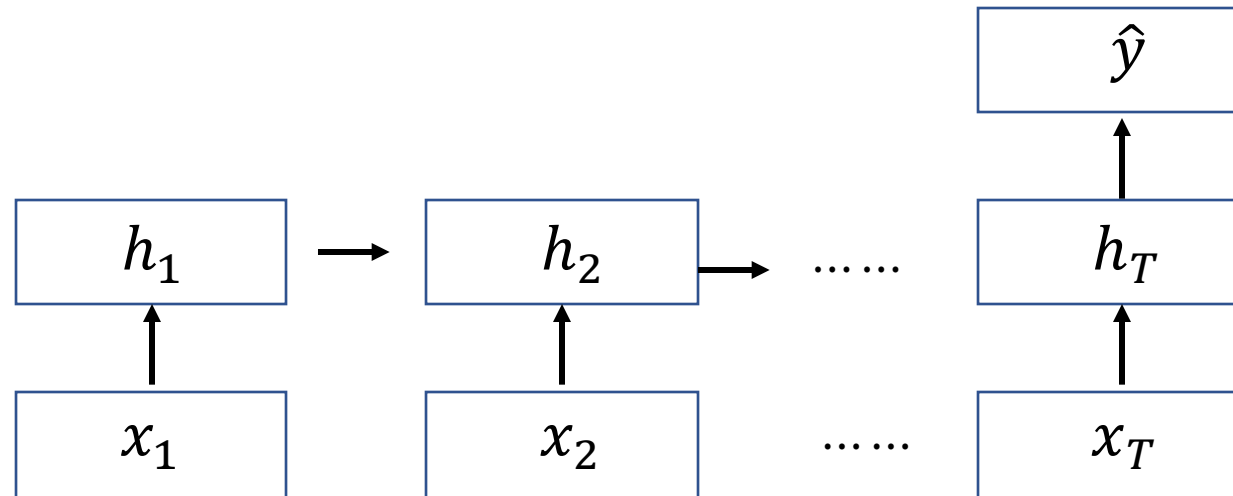
e.g., **Medical video classification on frame level**

Sequence to class: case (a)

For example, in the text process, the input is a text sequence, and its output is the class of this text.

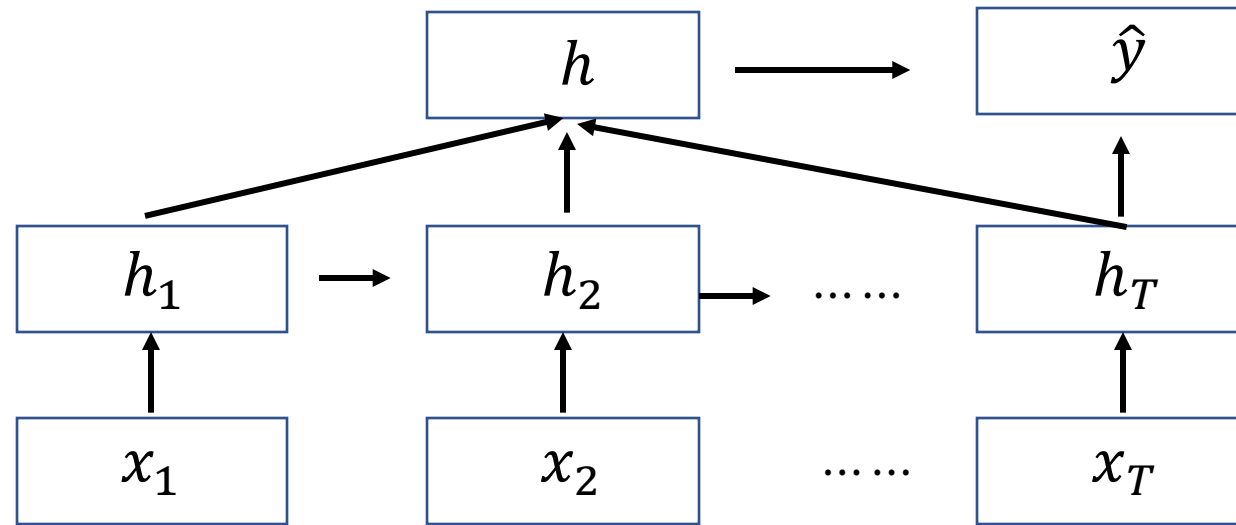
$$\hat{y} = g(h_T)$$

g can be the simple logistic regression or DNN.



Case (a)

Sequence to class (case b)

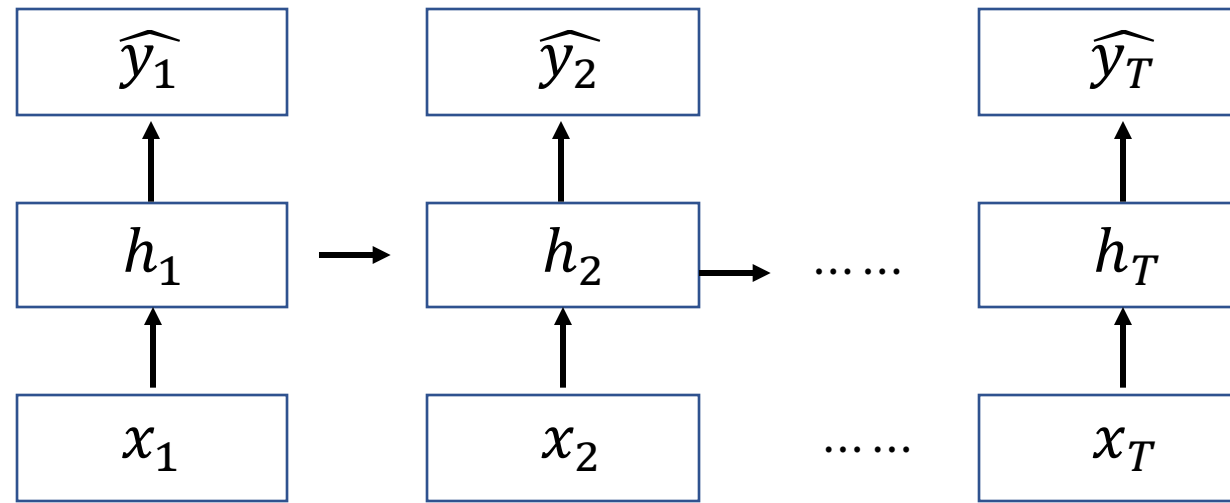


Case (b)

Sequence to class: case (b)

$$\hat{y} = g\left(\frac{1}{T} \sum_{t=1}^T h_t\right)$$

Synchronized seq2seq pattern: sequence labeling task

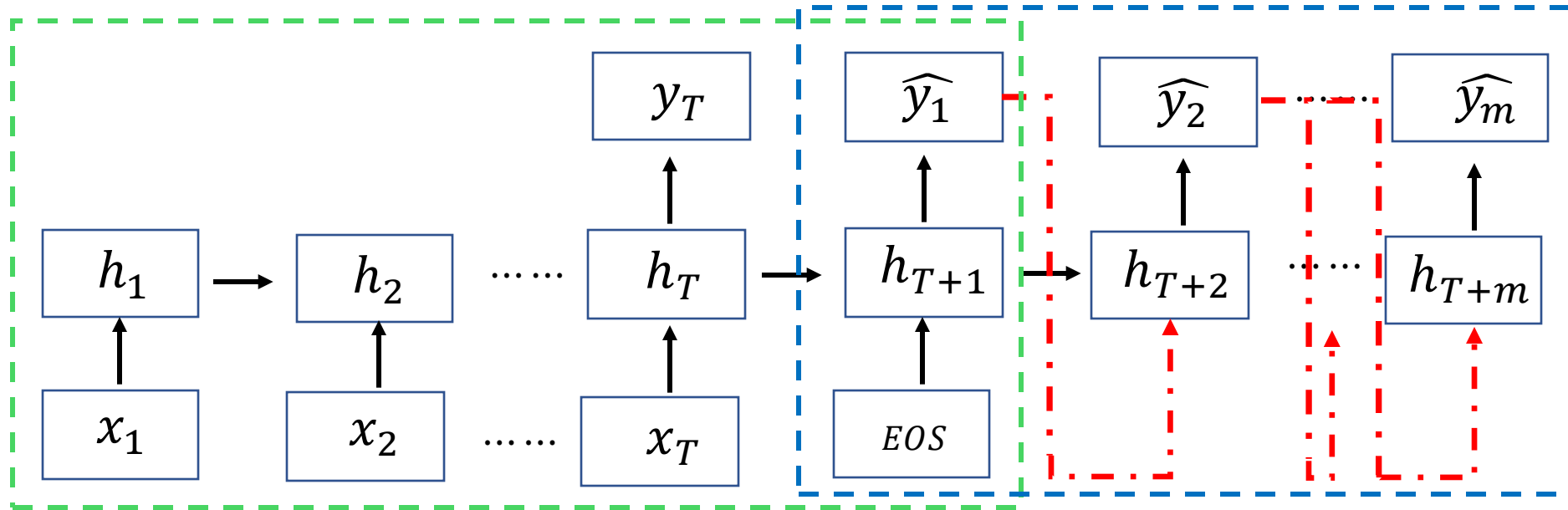


Synchronized seq2seq

$$\hat{y}_t = g(h_t) \text{ for } t \text{ in } [1, \dots, T].$$

$$h_t = f_W(h_{t-1}, x_t)$$

Desynchronized seq2seq pattern: encoder-decoder model



Here, T not necessary equals to m . This model has two RNNs, one is for the encoder, and the other is for the decoder.

$$\begin{aligned} h_t &= RNN_{encoder}(h_{t-1}, x_t), \forall t \in [1, \dots, T], \\ h_{T+t} &= RNN_{decoder}(h_{T+t-1}, \widehat{y_{t-1}}), \forall t \in [1, \dots, M], \\ \widehat{y_t} &= g(h_{T+t}), \forall t \in [1, \dots, M]. \end{aligned}$$

Backpropagation through time (BPTT)

Backpropagation Through Time

- Without loss of generality, we take the case 2: synchronized seq2seq RNN as an example.
- Given one training sample (x, y) , where $x_{1:T} = (x_1, \dots, x_T)$ is the input sequence, and $y_{1:T} = (y_1, \dots, y_T)$ is the label sequence.
- Then we have the loss at time step t : $L_t = L(y_t, g(h_t))$, here $g(h_t)$ is the output at time step t .

For the whole sequence, the loss is

$$L = \sum_{t=1}^T L_t.$$

Thus

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \frac{\partial L_t}{\partial U}.$$

Note that: $z_k = Uh_{k-1} + Wx_k + b$; $h_k = f(z_k)$, f is activation function.

Backpropagation Through Time

Now, let's go to the BPTT (**B**ackpropagation **T**hrough **T**ime).

1. Firstly, let's compute $\frac{\partial L_t}{\partial U}$, since $z_k = Uh_{k-1} + Wx_k + b$, thus

$$\frac{\partial L_t}{\partial u_{ij}} = \sum_{k=1}^t \boxed{\frac{\partial L_t}{\partial z_k}} \cdot \boxed{\frac{\partial z_k}{\partial u_{ij}}} \quad z_k \in R^m; U \in R^{m \times n}; h \in R^n$$

$$\frac{\partial z_k}{\partial u_{ij}} = [0, \dots, [h_{k-1}]_j, 0 \dots] \Leftrightarrow \Pi_i([h_{k-1}]_j)$$

$\downarrow$
 i

Backpropagation Through Time

Now we focus on the **error item** $\frac{\partial L_t}{\partial z_k}$. Note that $z_k = Uh_{k-1} + Wx_k + b$.

When $1 \leq k \leq t$,

$$\frac{\partial L_t}{\partial z_k} = \boldsymbol{\delta}_{t,k} = \frac{\partial L_t}{\partial z_{k+1}} \cdot \frac{\partial z_{k+1}}{\partial h_k} \cdot \frac{\partial h_k}{\partial z_k} = \delta_{t,k+1} \cdot U^T \cdot \text{diag}(f'(z_k))$$

$$\frac{\partial L_t}{\partial u_{ij}} = \sum_{k=1}^t \frac{\partial L_t}{\partial z_k} \cdot \frac{\partial z_k}{\partial u_{ij}} = \sum_{k=1}^t [\delta_{t,k}]_i \cdot [h_{k-1}]_j.$$

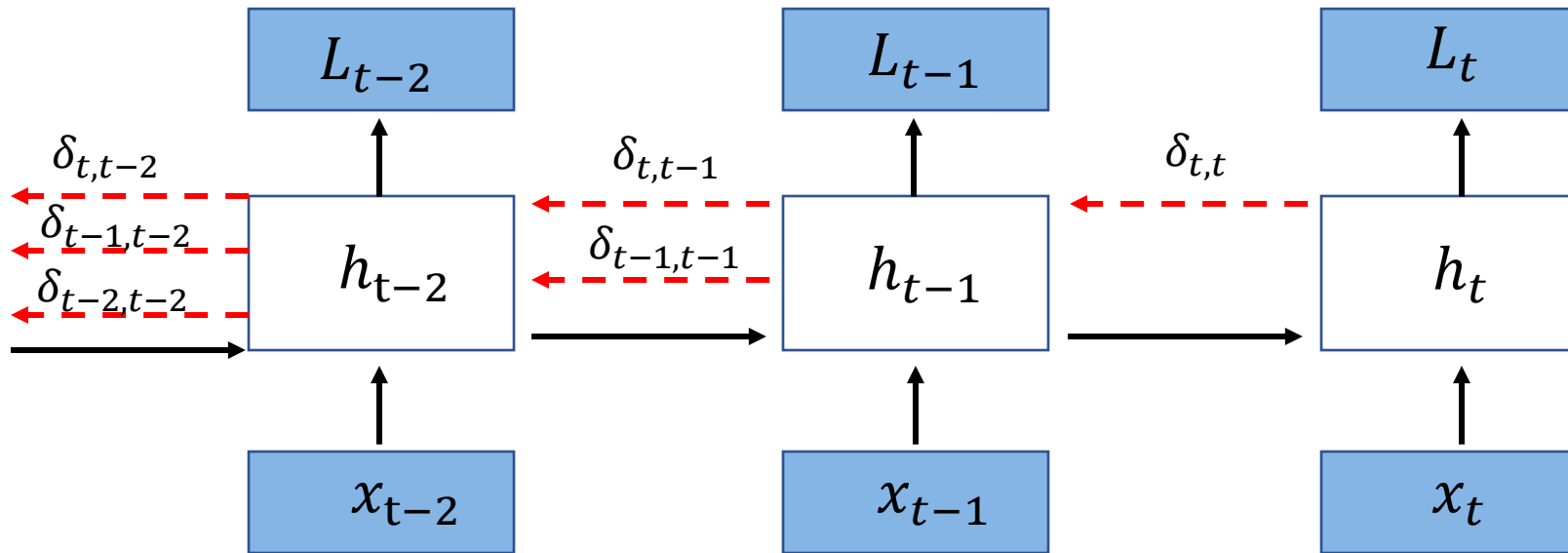
In matrix form:

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \cdot h_{k-1}^T$$

In the same manner:

$$\frac{\partial L_t}{\partial W} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \cdot x_k^T$$

$$\frac{\partial L_t}{\partial b} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}$$



Error items backpropagate through time

Gradient vanishing/exploding in RNNs

- Before showing the vanishing/exploding gradient problem, we first give motivation: why we care?
- Because long term dependency requires gradient to flow back “fluently” long-range through time
- Example:

Jane walked into the room. John walked in too. It was late in the Day. Jane said hi to _____

The problem of gradients exploding or vanishing

- According to the $\delta_{t,k} = \delta_{t,k+1} \cdot U^T \cdot \text{diag}(f'(z_k))$ in the previous introduction of BPTT, we obtained:

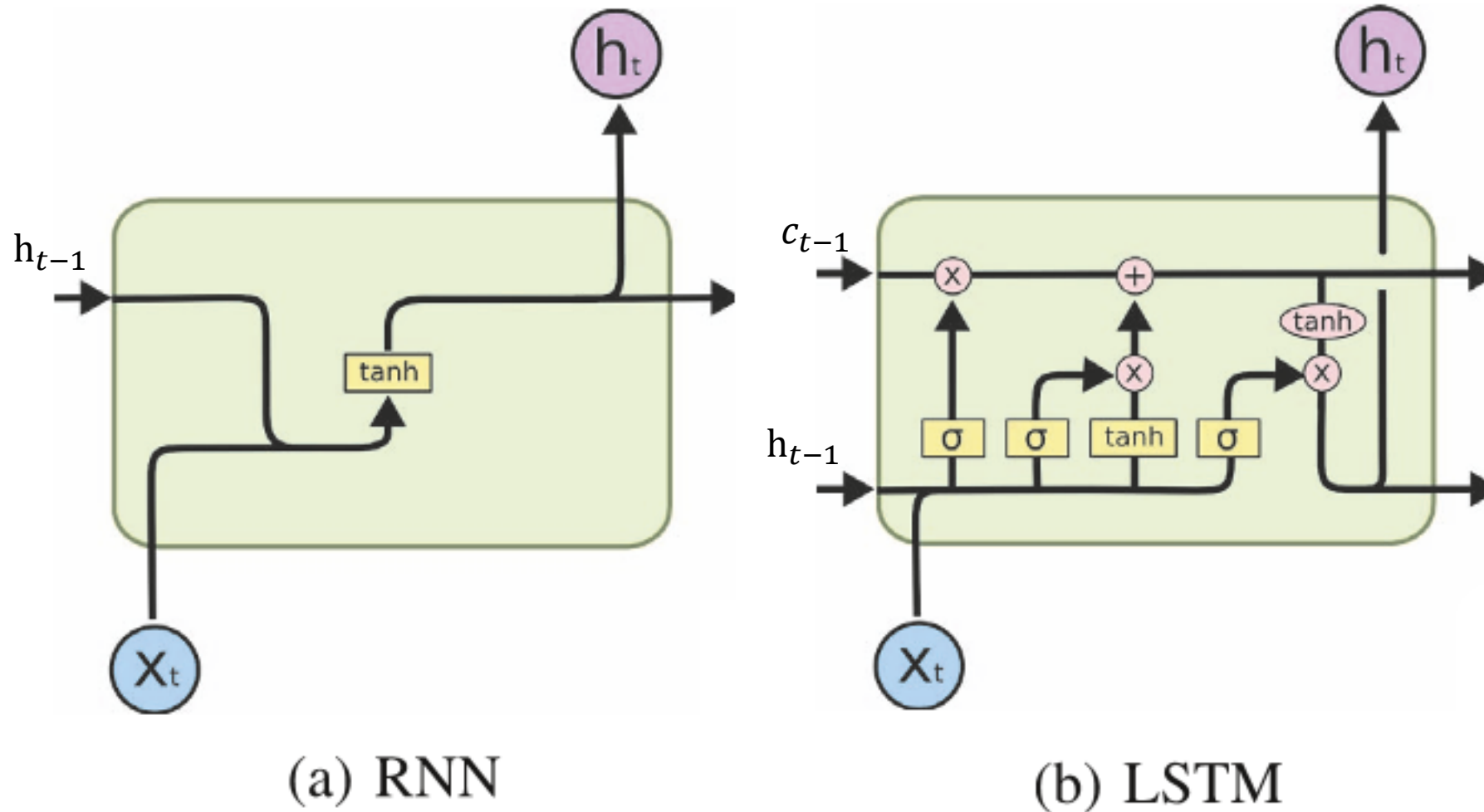
$$\delta_{t,k} = \prod_{\tau=k}^{t-1} (\text{diag}(f'(z_\tau))U^T) \cdot \delta_{t,t}, \quad k \leq t$$

- Define $\gamma \cong \left\| \text{diag}(f'(z_\tau))U^T \right\|$, then $\delta_{t,k} = \gamma^{t-k} \cdot \delta_{t,t}$.

If $\gamma > 1$, when $t - k$ is too large, γ^{t-k} will be too large, **gradient explodes** after some multiplications.

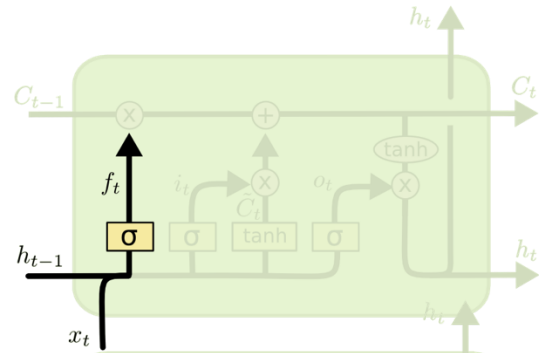
If $\gamma < 1$, when $t - k$ is too large, γ^{t-k} will be too small, **gradient vanishes** after some multiplications.

Long Short-Term Memory (LSTM)

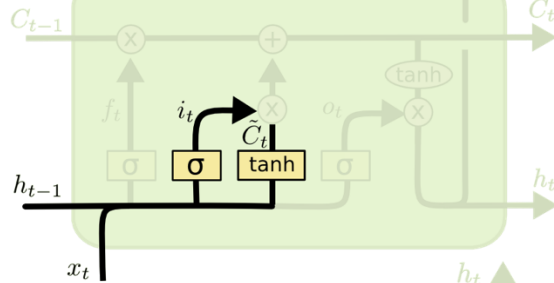


It adds some gate mechanism, and new inner states c_t .

LSTM

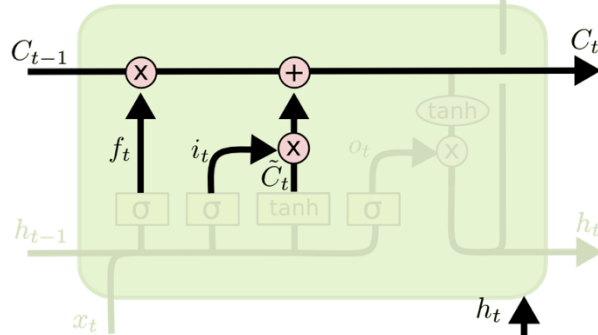


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

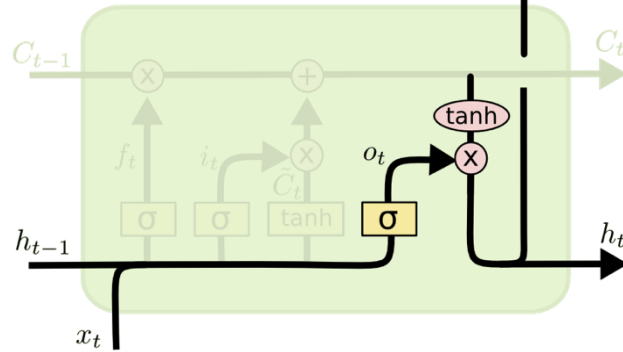


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

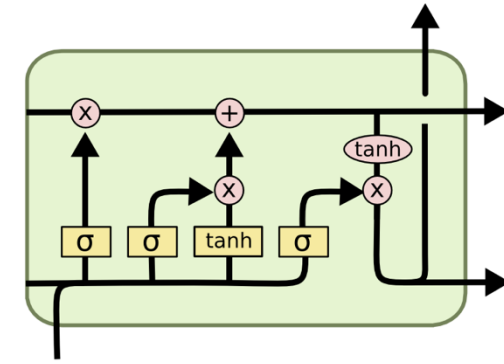


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

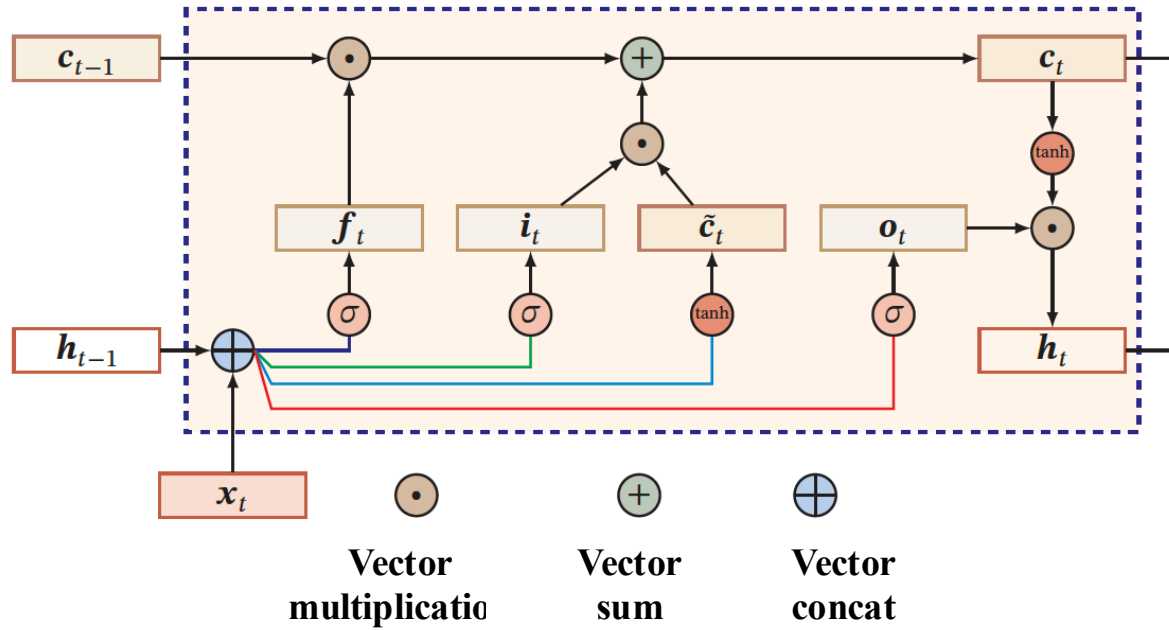
$$h_t = o_t * \tanh(C_t)$$



i_t decides what component is to be updated.
 $\tilde{C}_t$ provides change contents

Updating the cell state

Decide what part of the cell state to output



Overview of the LSTM

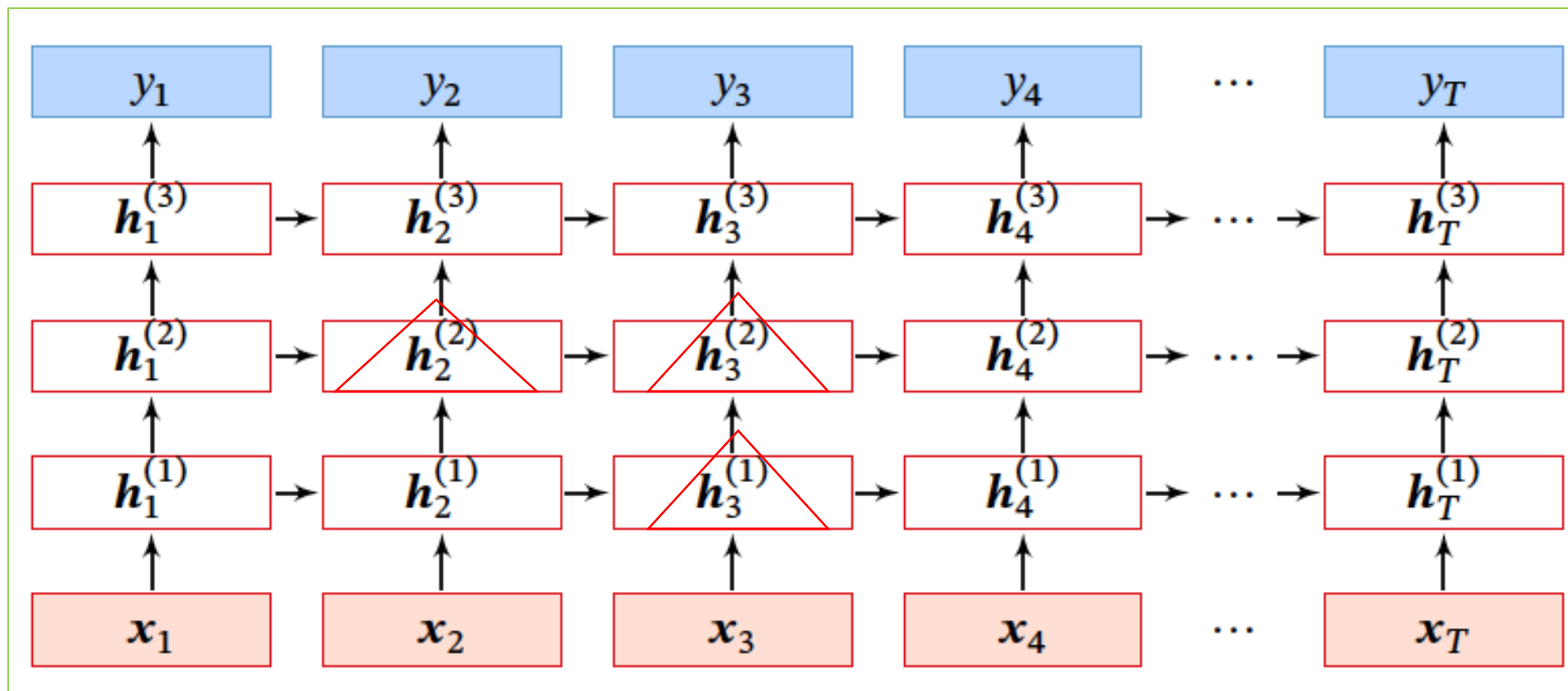
$$\begin{bmatrix} \tilde{c}_t \\ o_t \\ i_t \\ f_t \end{bmatrix} = \begin{bmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{bmatrix} \left(W \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix} + b \right),$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$h_t = o_t \odot \tanh(c_t),$$

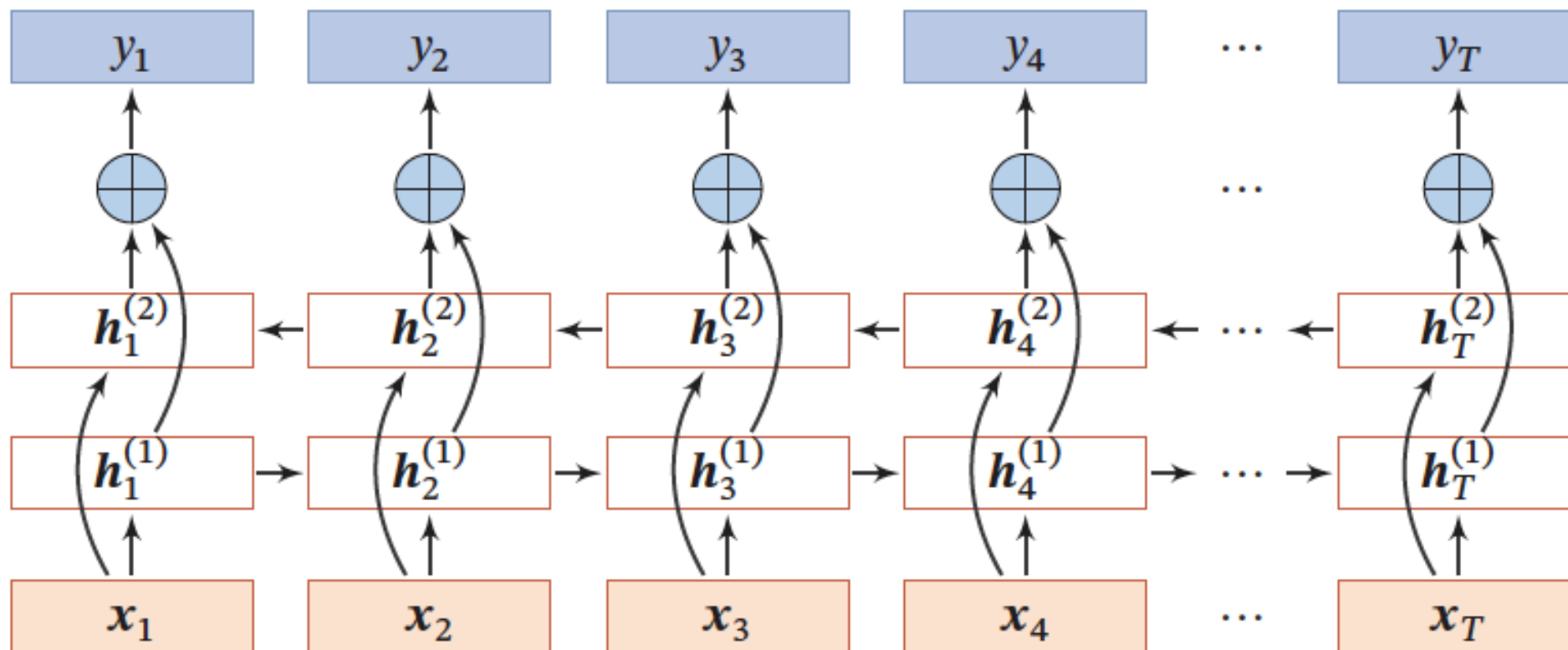
Deep layer RNN

Deep stacked RNN



$$h_t^{(l)} = f(U^{(l)}h_{t-1}^{(l)} + W^{(l)}h_t^{(l-1)} + b^{(l)}),$$

Bidirectional RNN



$$h_t^{(1)} = f(U^{(1)}h_{t-1}^{(1)} + W^{(1)}x_t + b^{(1)}),$$

$$h_t^{(2)} = f(U^{(2)}h_{t+1}^{(2)} + W^{(2)}x_t + b^{(2)}),$$

$$h_t = h_t^{(1)} \oplus h_t^{(2)},$$

Attention and Transformer

Can transformer replace CNN?

Transformers are replacing CNNs as the **default architecture for new vision systems**, particularly when:

1. Training data $> 10\text{M}$ images
2. Global context is critical
3. Multimodal integration is required

However, CNNs remain **unreplaced for**:

- Ultra-low-power devices
- Specialized texture analysis
- Systems requiring minimal retraining from existing pipelines

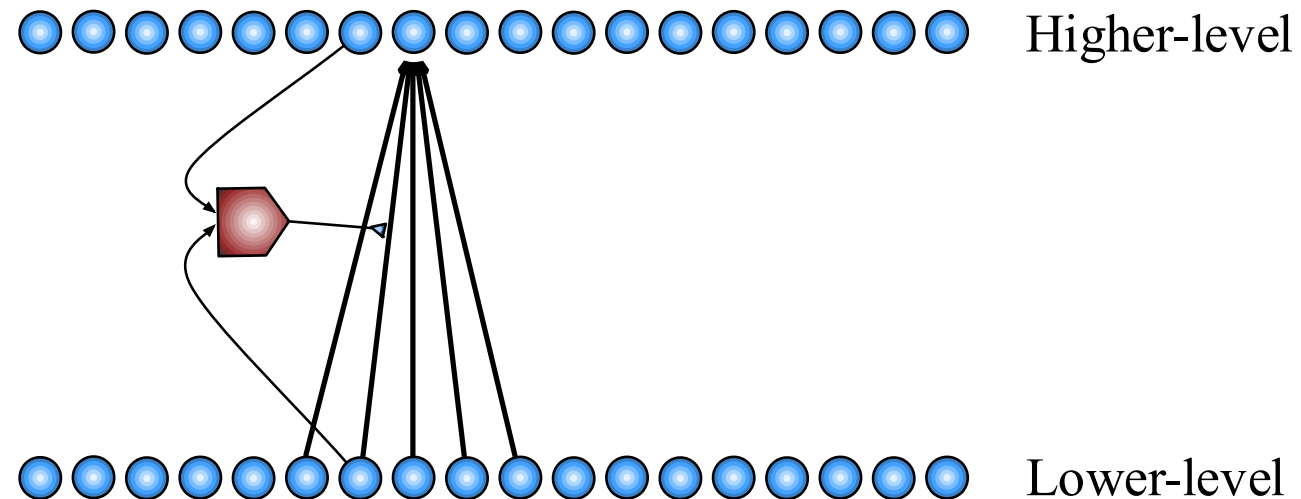
Attention Mechanism for Deep Learning



Attention Mechanism for Deep Learning

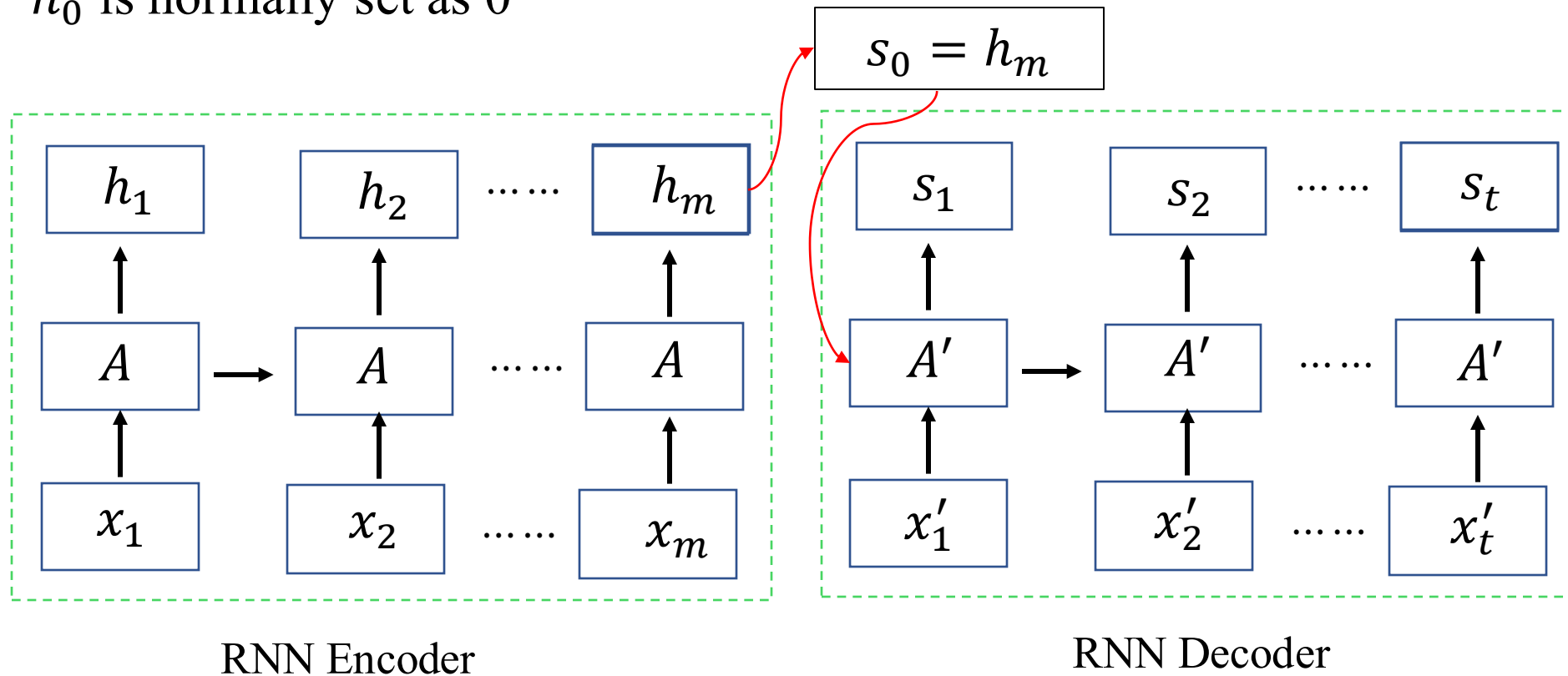
- Consider an input (or intermediate) sequence or image
- Consider an upper-level representation, which can choose “where to look”, by assigning a weight or probability to each input position, applied at each position.

Softmax over lower locations conditioned on the context at lower and higher locations



Recall RNN-based Encoder-Decoder Model

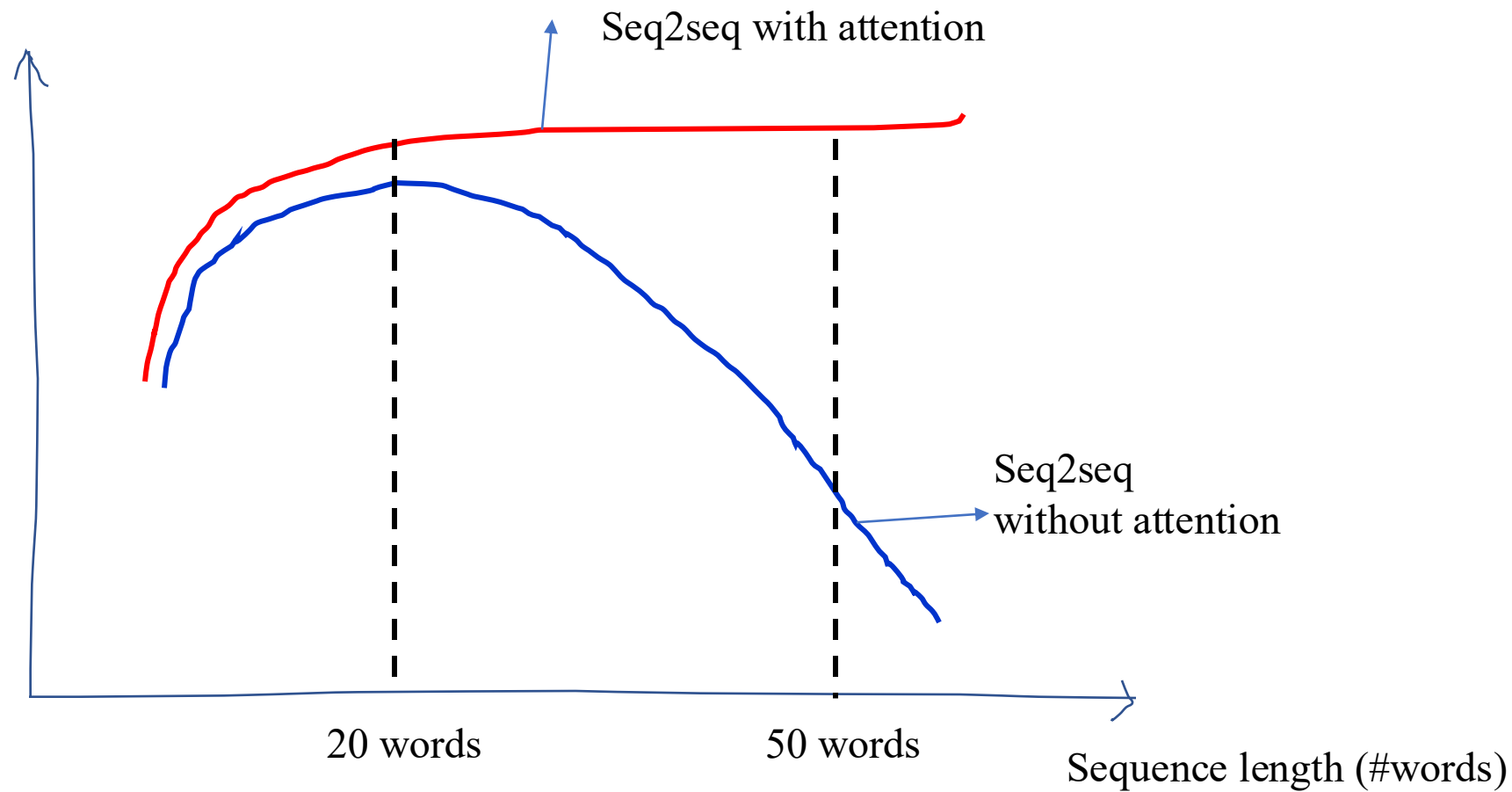
h_0 is normally set as 0



Even h_m includes all the input information $(x_1, x_2, \dots, x_m)$. The shortcoming of the above seq2seq model is that the final state cannot remember a long sequence.

$$s_1 = \tanh(A' \begin{bmatrix} x'_1 \\ s_0 \end{bmatrix} + b)$$

BLEU (Bilingual Evaluation Understudy)



- The **BLEU** evaluation metric measures the performance of a machine translation system by calculating the matching degree between the translations generated by the machine and the human translations.
- The calculation: 1) it computes the number of matches between n-grams (typically taken as 1-grams to 4-grams) in the machine translation and the human reference translations, divided by the total number of n-grams in the machine translation; 2) it takes the geometric mean of the match rates for all n-grams.

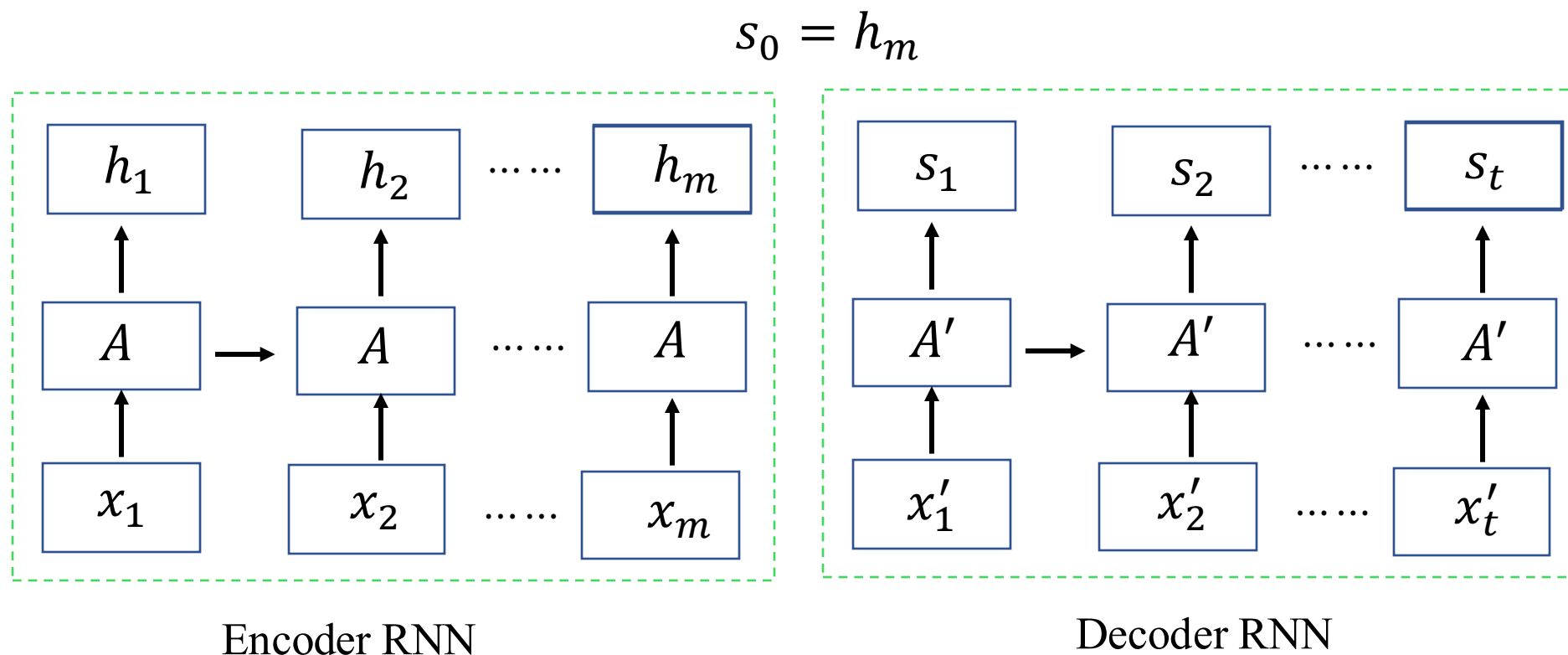
Attention mechanism

It was proposed in 2015 ICLR conference:

- With attention, Seq2seq model does not forget source input;
- With attention, the decoder knows where to focus;
- However, attention brings much more computation cost.

Neural Machine Translation by Jointly Learning to Align and Translate,
<https://arxiv.org/pdf/1409.0473.pdf>

Attention mechanism



$$s_0 = h_m$$
$$s_1 = \tanh\left(A' \begin{bmatrix} x'_1 \\ s_0 \\ c_0 \end{bmatrix} + b\right)$$

By calculating the correlation between s_0 and h_i , we can calculate c_0 (called context vector):

$$c_0 = \alpha_1 h_1 + \alpha_2 h_2 + \cdots + \alpha_m h_m$$

How to calculate α_i ?

How to calculate α_i ?

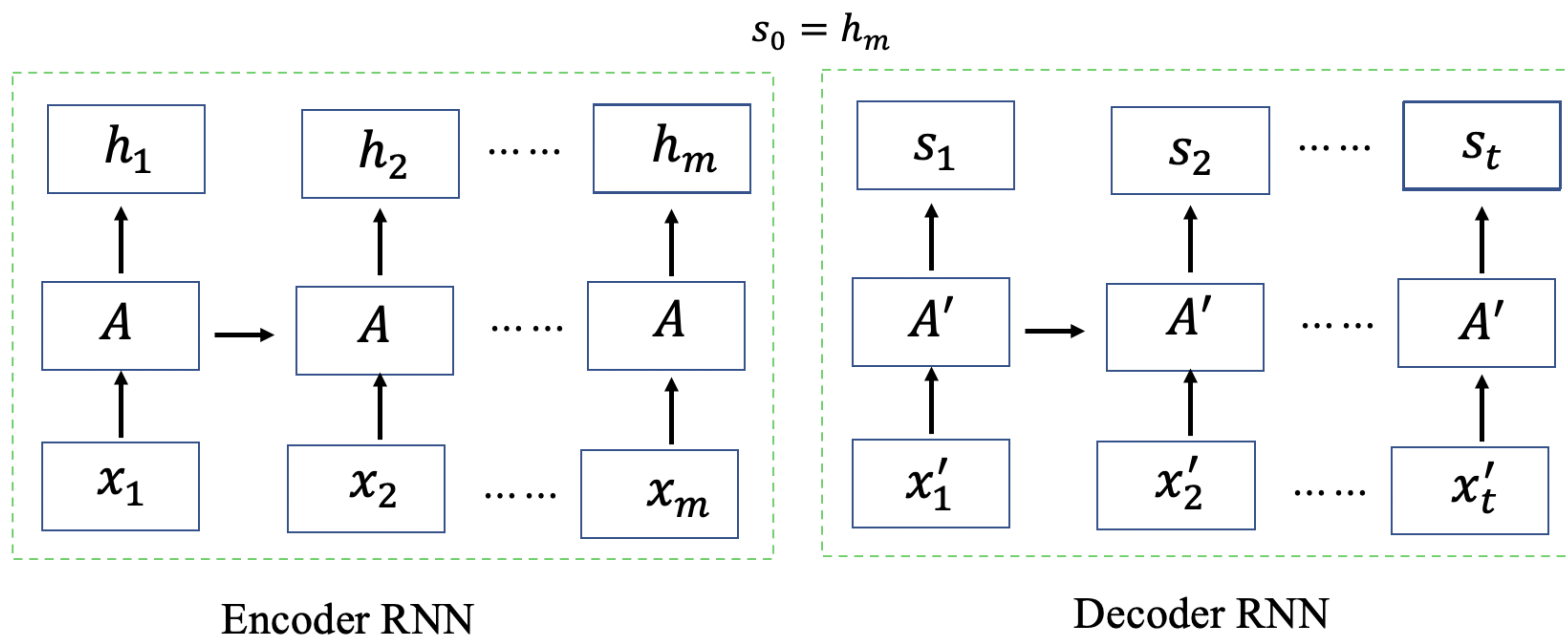
There are two options:

Option 1: used in the original paper (ICLR 2015)

$$\tilde{\alpha}_i = V^T \cdot \tanh(W[h_i, s_0]), \text{ where } V \text{ and } W \text{ are trainable parameters.}$$

Then normalize $\tilde{\alpha}_1, \tilde{\alpha}_2, \dots, \tilde{\alpha}_m$ so that they sum to 1.

$$[\alpha_1, \alpha_2, \dots, \alpha_m] = \text{Softmax}([\tilde{\alpha}_1, \tilde{\alpha}_2, \dots, \tilde{\alpha}_m]).$$



How to calculate α_i ?

Option 2: this is more popular and the transformer uses this option.

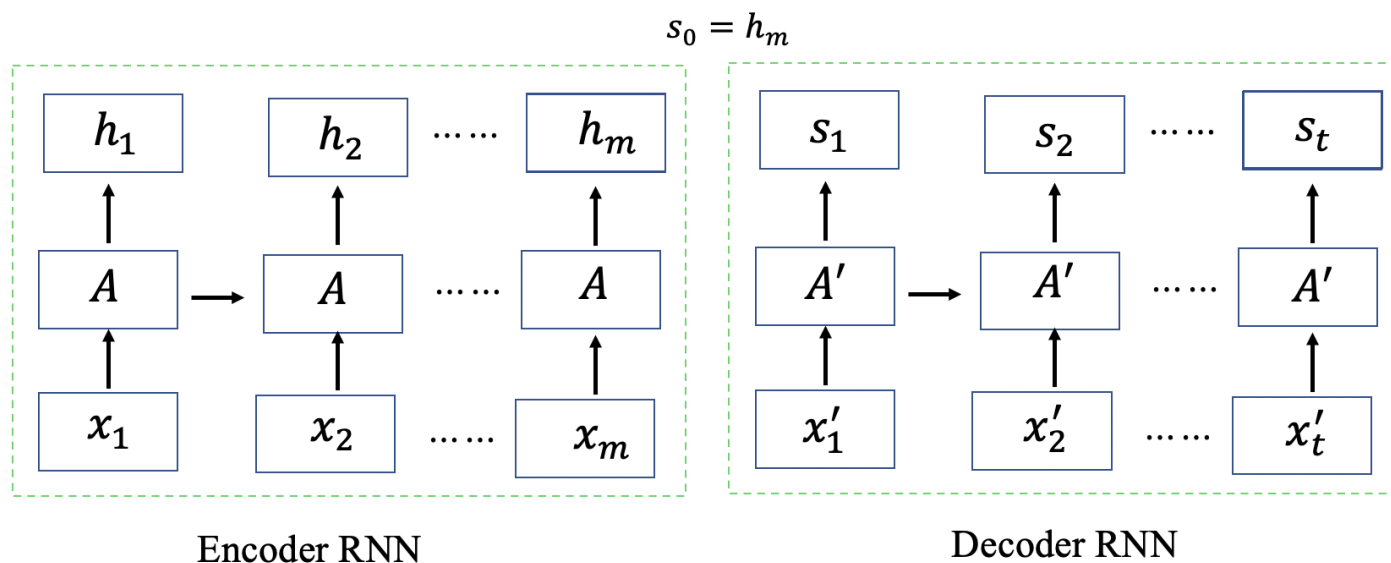
1. Linearly transform the h_i, s_0 :

$$k_i = W_k \cdot h_i \text{ for } i \in [1, \dots, m];$$

$$q_0 = W_Q \cdot s_0.$$

2. Calculating $\tilde{\alpha}_i = k_i^T q_0$ for $i \in [1, \dots, m]$;

3. Normalization: $[\alpha_1, \alpha_2, \dots, \alpha_m] = \text{Softmax}([\tilde{\alpha}_1, \tilde{\alpha}_2, \dots, \tilde{\alpha}_m])$.

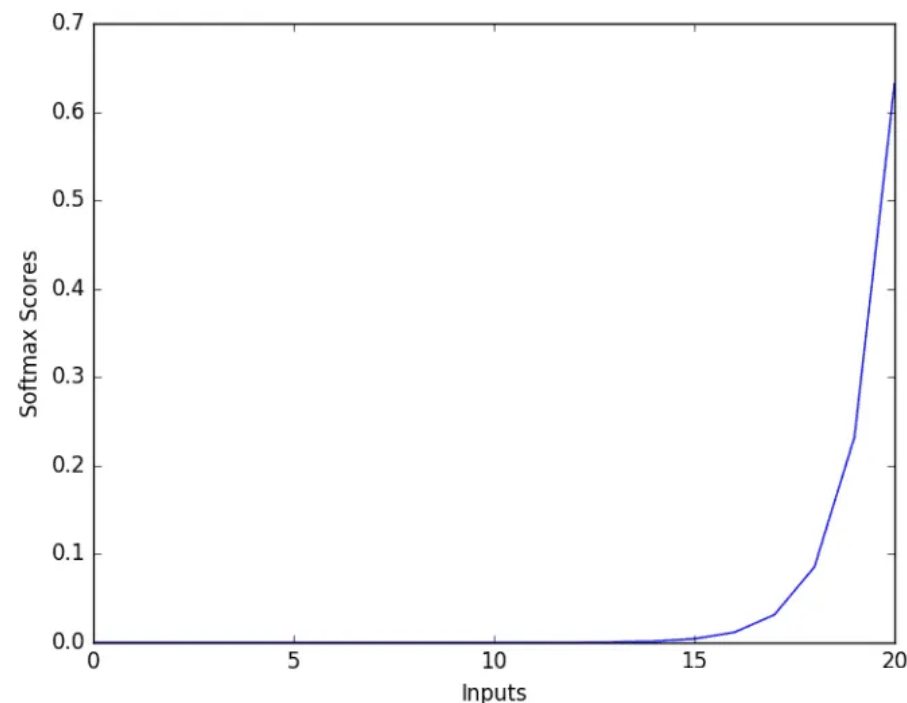


$$\text{attention}(Q, K, V) = \text{Softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V$$

How to calculate α_i ?

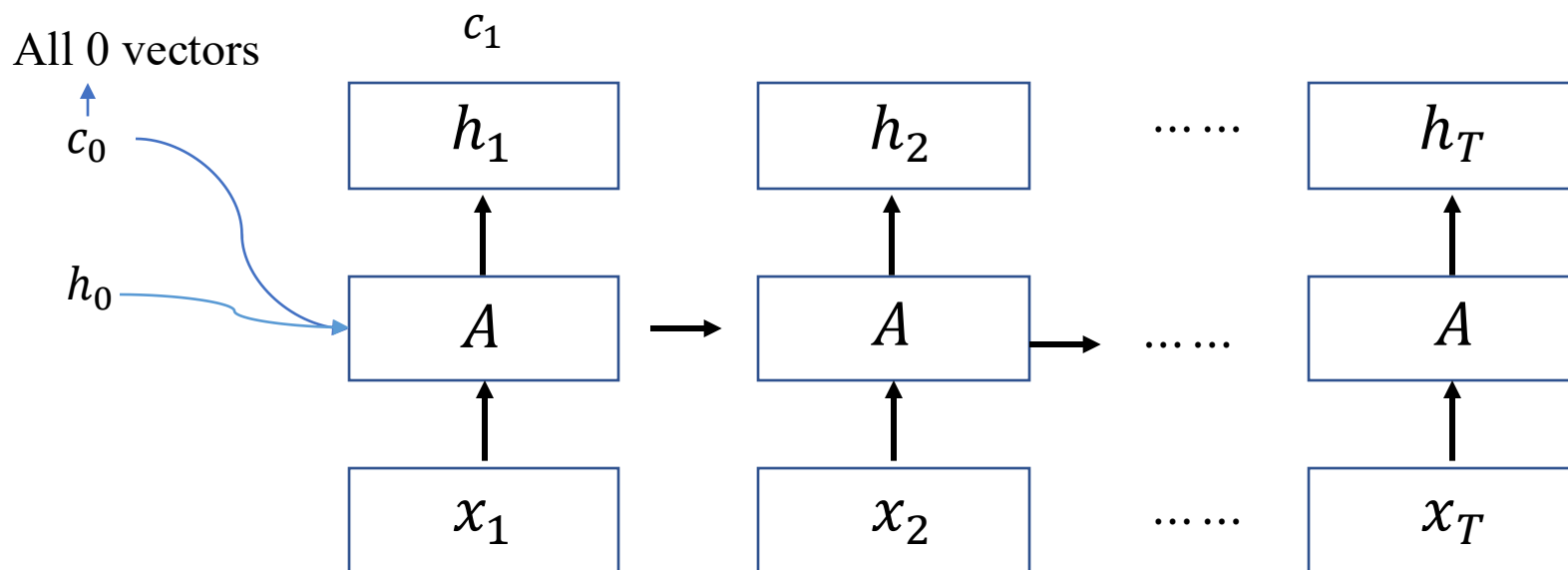
$$\text{attention}(Q, K, V) = \text{Softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V$$

1. To prevent input values into softmax from becoming too large, which could lead to abnormal derivatives.
2. Choosing the square root of d_k is because it helps ensure that the result of $Q \times K$ follows a distribution with mean of 0 and a variance of 1, similar to normalization.



Self-attention mechanism

Let's recalled that the attention is used with RNN seq2seq model. Here, we introduce the **Self-attention mechanism**, which can be used with all kinds of RNN, not necessary with the seq2seq model.



Simple RNN

$$h_1 = \tanh\left(A \begin{bmatrix} x_1 \\ h_0 \end{bmatrix} + b\right)$$

Simple RNN + Self attention

$$h_1 = \tanh\left(A \begin{bmatrix} x_1 \\ c_0 \end{bmatrix} + b\right) \text{ or } \tanh\left(A \begin{bmatrix} x_1 \\ c_0 \\ h_0 \end{bmatrix} + b\right)$$

How to calculate c_i ?

Let's see the case for c_2 .

Firstly, calculating weight $\alpha_i = \text{align}(h_i, h_2)$, for $i = 1, 2$

Thus:

$$c_2 = \alpha_1 h_1 + \alpha_2 h_2.$$

The same manner for $c_2, c_3, \dots$

Summary

- With self-attention, RNN is less to forget. Attention and Self-attention are very similar, but Self-attention is not limited to Seq2seq model.
- It pays attention to the context relevant to the new input.

Transformer

Firstly, we introduce the Transformer without using RNN, only with attention (2017 Neurips: “**Attention is all you need**” Cited by 133960).

- Transformer is a Seq2seq model
- Transformer is not RNN
- Purely based on the attention and dense layer
- Shows higher performance than RNNs on large datasets.

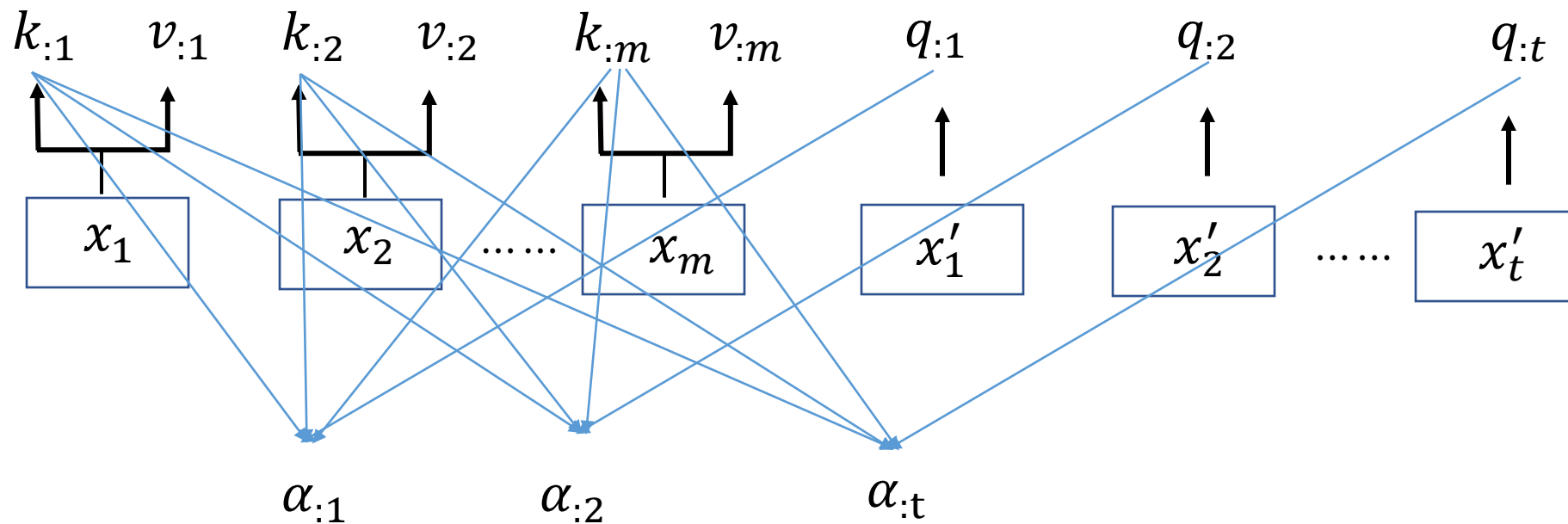
Keys and values are based on **encoder**'s input $x_1, x_2, \dots, x_m$.

Key: $k_{:i} = W_k x_i$; Value: $V_{:i} = W_v x_i$.

Queries are based on the **decoder**'s inputs $x'_1, x'_2, \dots, x'_t$

Query: $q_{:i} = W_Q x'_i$.

Transformer



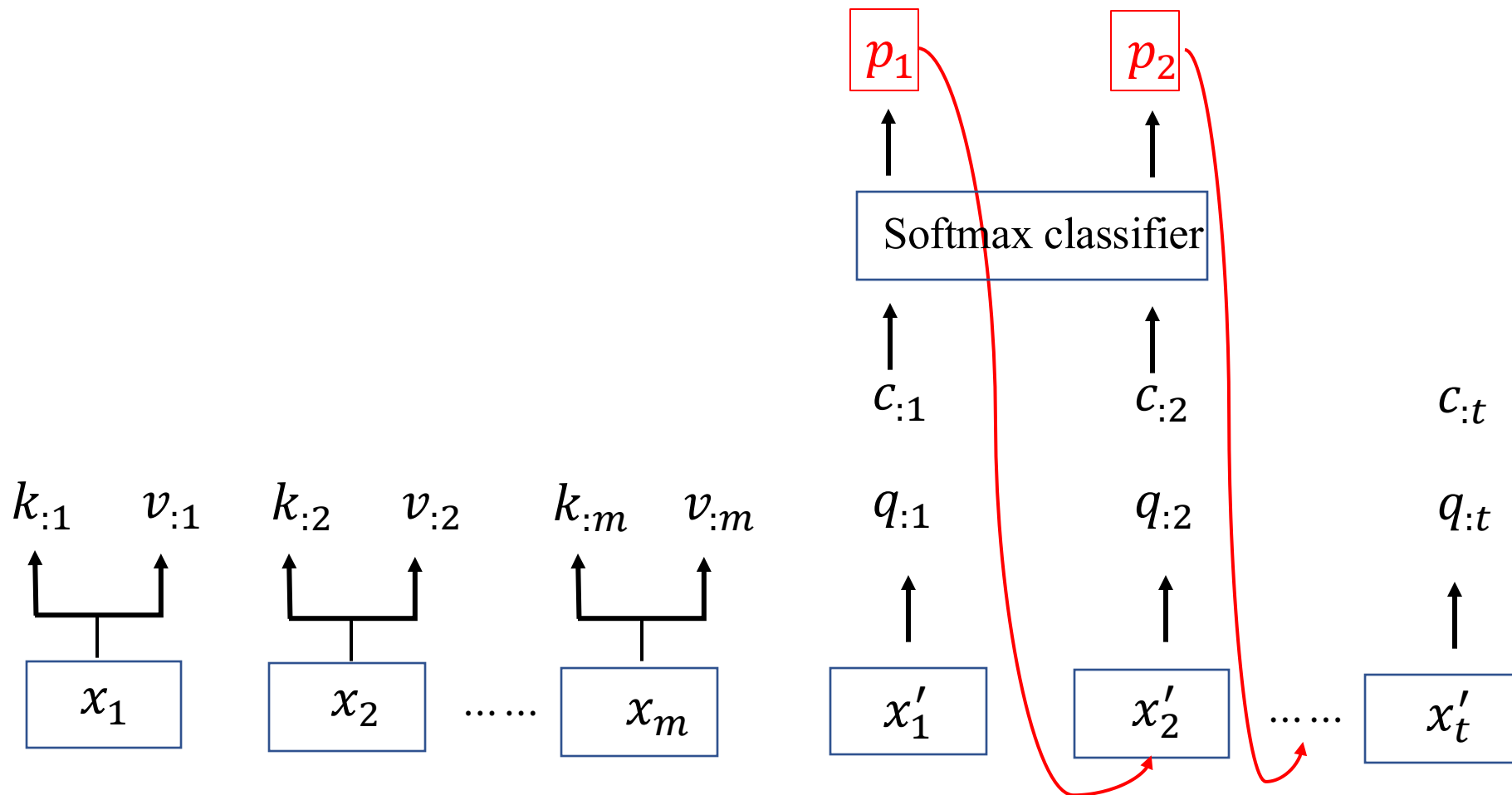
$$\alpha_{:1} = \text{softmax}(K^T q_{:1} / \sqrt{d_k}) \in R^m$$

$$c_{:1} = \alpha_{11} v_{:1} + \alpha_{21} v_{:2} + \dots + \alpha_{m1} v_{:m} = V \alpha_{:1}$$

Output of attention layer: $C = [c_{:1}, c_{:2}, \dots, c_{:t}]$.

$c_{:j} = V \cdot \text{softmax}(K^T q_{:j})$ thus $c_{:j}$ is related with x'_j and $[x_1, x_2, \dots, x_m]$.

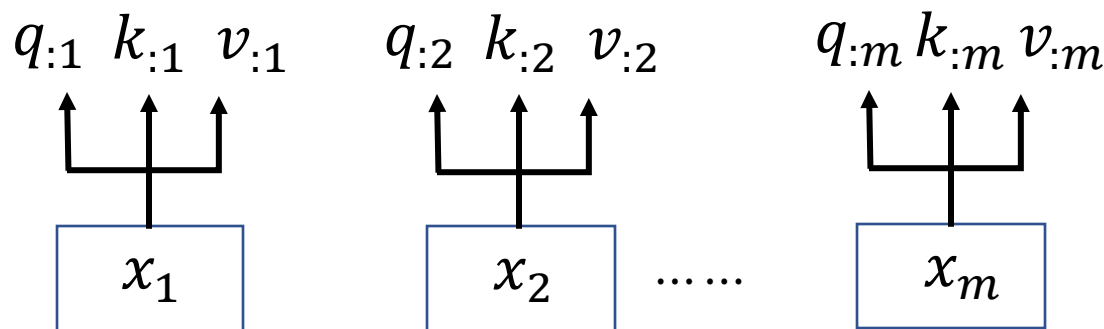
Transformer testing stage



In this case, attention is used to replaced RNN.

Self-attention without RNN

Self-attention layer $C = \text{Atten}(X, X)$, $X = [x_1, x_2, \dots, x_m]$



Keys, values and Queries are all based on input $x_1, x_2, \dots, x_m$.

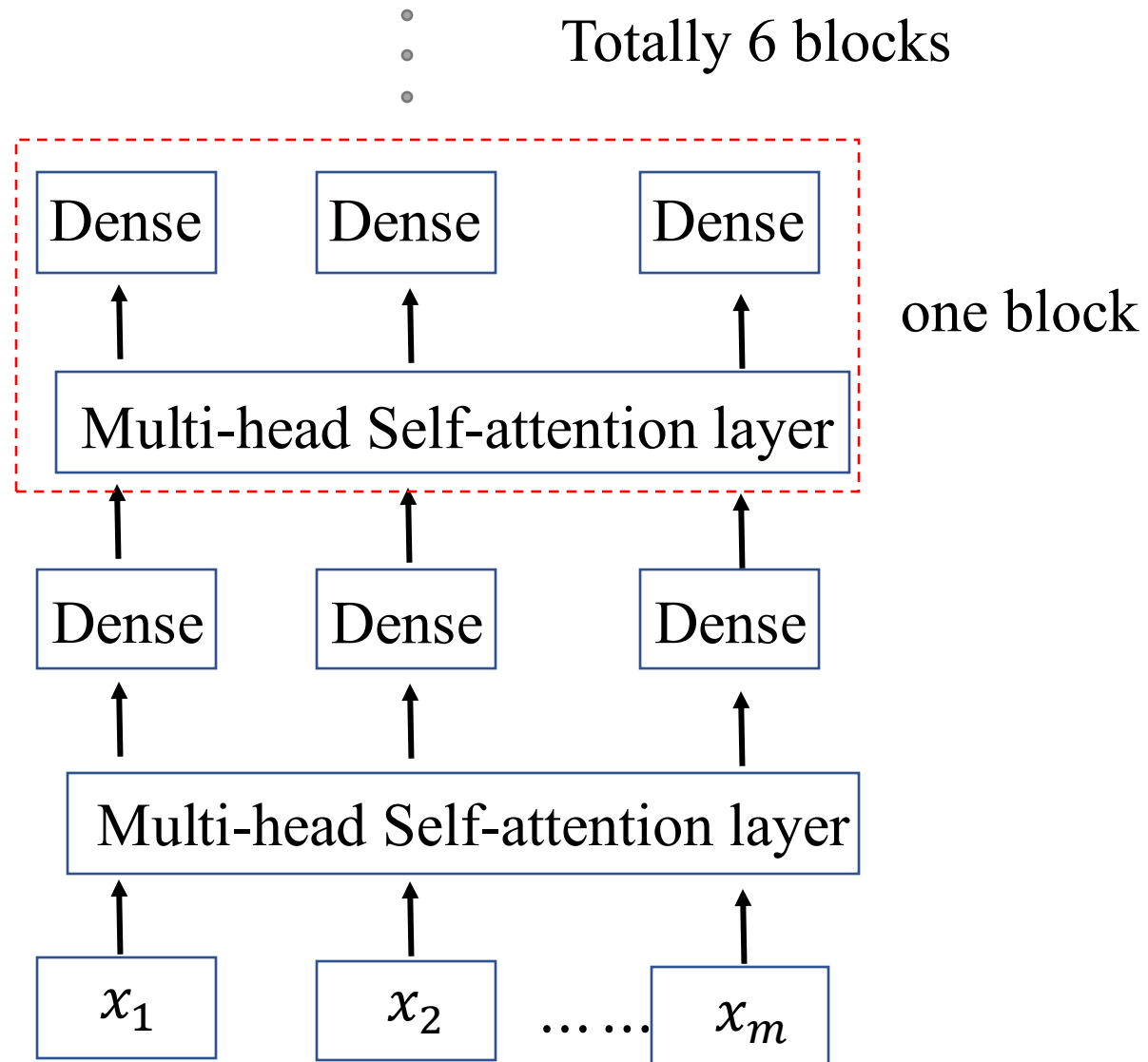
Key: $k_{:i} = W_k x_i$; **Value:** $V_{:i} = W_v x_i$; **Query:** $q_{:i} = W_Q x_i$.

Weight $\alpha_{:j} = \text{softmax}(K^T q_{:j} / \sqrt{d_k}) \in R^m$

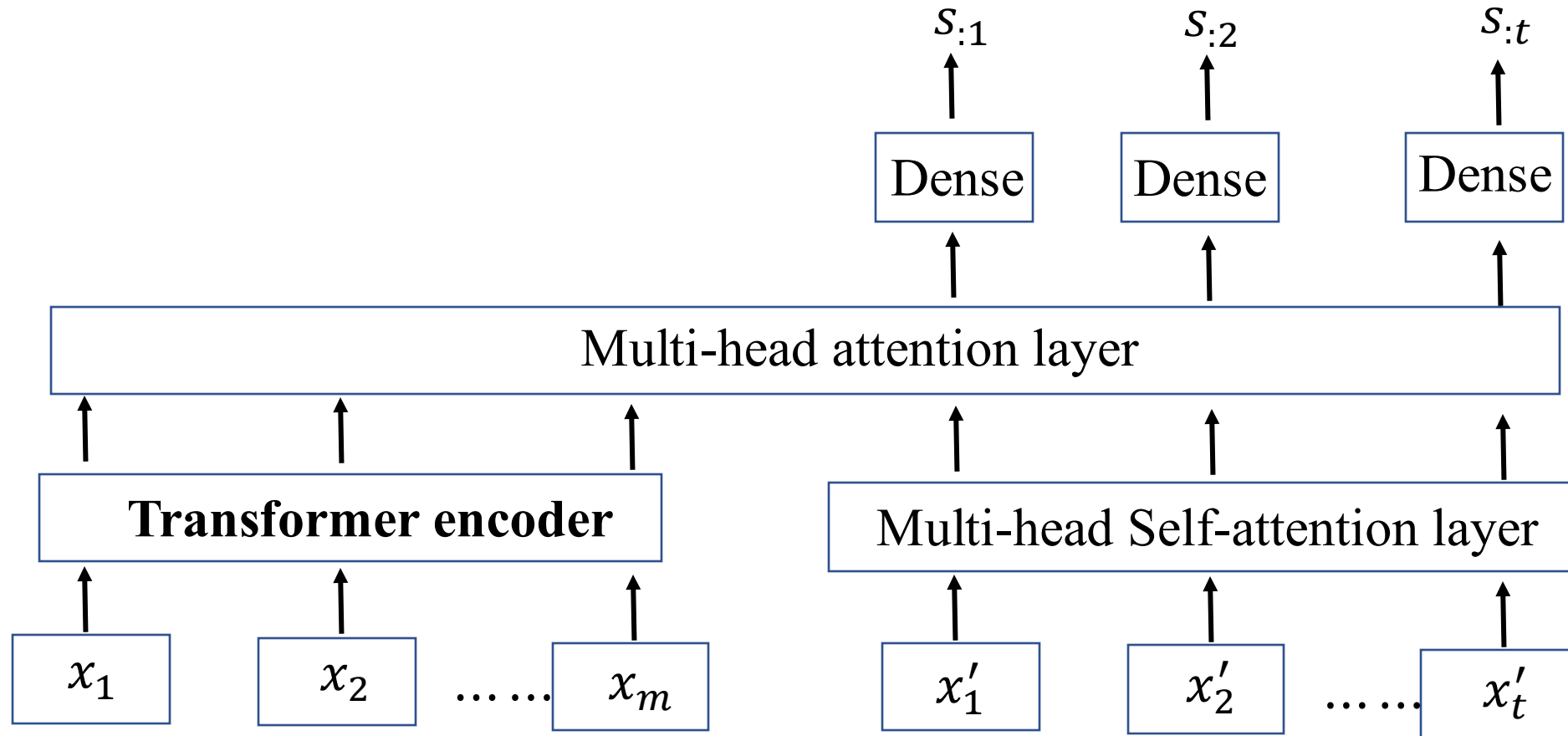
$$c_{:j} = \alpha_{1j} v_{:1} + \alpha_{2j} v_{:2} + \dots + \alpha_{mj} v_{:m} = V \alpha_{:j}$$

$c_{:j}$ is dependent on V , and $\alpha_{:j}$.

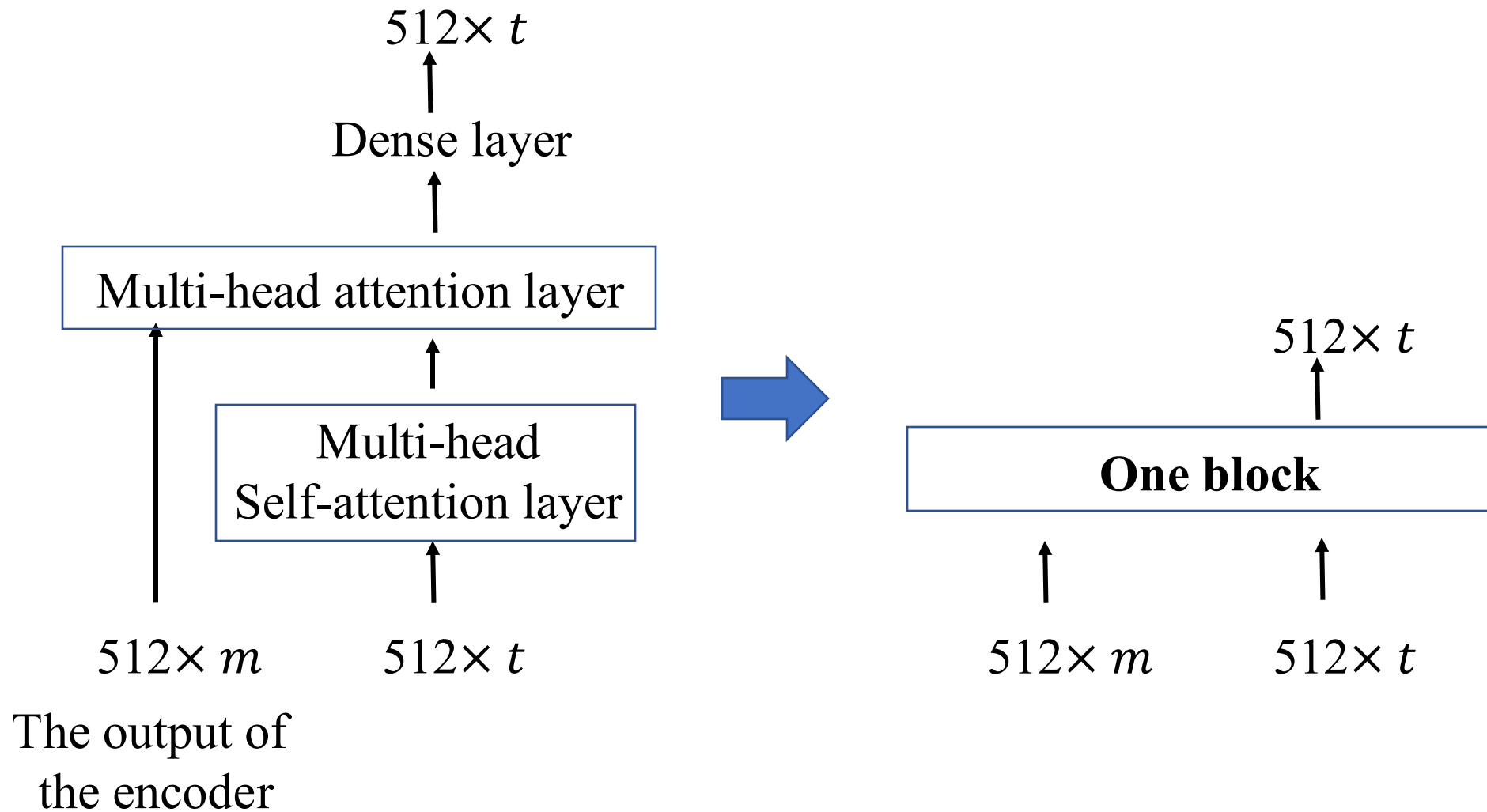
Encoder of Transformer



Decoder of Transformer



This is one block



Basic diagram of the Transformer has six such blocks.

Why Do We Need Positional Encoding in Transformer?

- The **Transformer** architecture, unlike RNNs, **does not have a built-in notion of sequence order**. The self-attention mechanism treats all tokens simultaneously and symmetrically, which makes it powerful but also order-agnostic.
- To give the model a sense of **token position in a sequence**, we add **positional encodings** to the input embeddings.

2. Two Main Types of Positional Encoding

2.1 Fixed (Sinusoidal) Positional Encoding

Introduced in the original paper by Vaswani et al. (2017), this method uses **sinusoidal functions** to generate position vectors.

For a position pos and embedding dimension i , the encoding is defined as:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Where:

- pos is the position in the sequence (e.g., 0, 1, 2...),
- i is the dimension index,
- d is the total embedding dimension.

✅ Advantages:

- No additional parameters.
- Generalizes to longer sequences not seen during training.
- Captures **relative distance** due to consistent frequency encoding.

✓ **Intuition Behind Sinusoidal Encoding:**

- Each dimension in the position vector corresponds to a sinusoid with a different frequency. This allows the model to **attend based on relative positions** between tokens.
- The difference between two positional vectors $PE(pos1) - PE(pos2)$ encodes relative distance in a smooth, regular way.

✓ **How Are Positional Encodings Used?**

- They are **added directly** to the input word embeddings before being fed into the Transformer:

$$\text{input_embedding} = \text{word_embedding} + \text{positional_encoding}$$

- The shape of the positional encoding must match the input embedding dimensions (*e.g.*, [sequence length, embedding dimension]).

2.2 Learnable Positional Embeddings

This method uses a **trainable embedding table**, similar to word embeddings:

```
python
```

```
pos_embedding = nn.Embedding(max_len, d_model)
```

Each position *pos* is assigned a learnable vector.

✅ Advantages:

- Allows the model to learn task-specific position representations.
- Often achieves better performance in practice, especially on large datasets.

Summary

- Attention was originally developed for Seq2seq RNNs model;
- Self-attention for all RNNs model (not necessary for Seq2seq models);
- Attention can be used without RNN;
- Until now, we know how to build attention layer and self-attention layer.

That is to say, Seq2seq model is not dependent on RNNs only anymore,

Attention layer: $C = \text{Atten}(X, X')$

Self-attention layer: $C = \text{Atten}(X, X)$

Summary

- Transformer is based on these two layers, but with multi-head.
- Multi-head is nothing but concatenate the output of all single-head attention. It allows the model to **jointly attend to information from different representation subspaces at different positions.**
- Will transformer suffers from gradient vanishing and exploding issues?

Let's see an example!

Sample Data: Tokenized Sentiment Analysis Examples

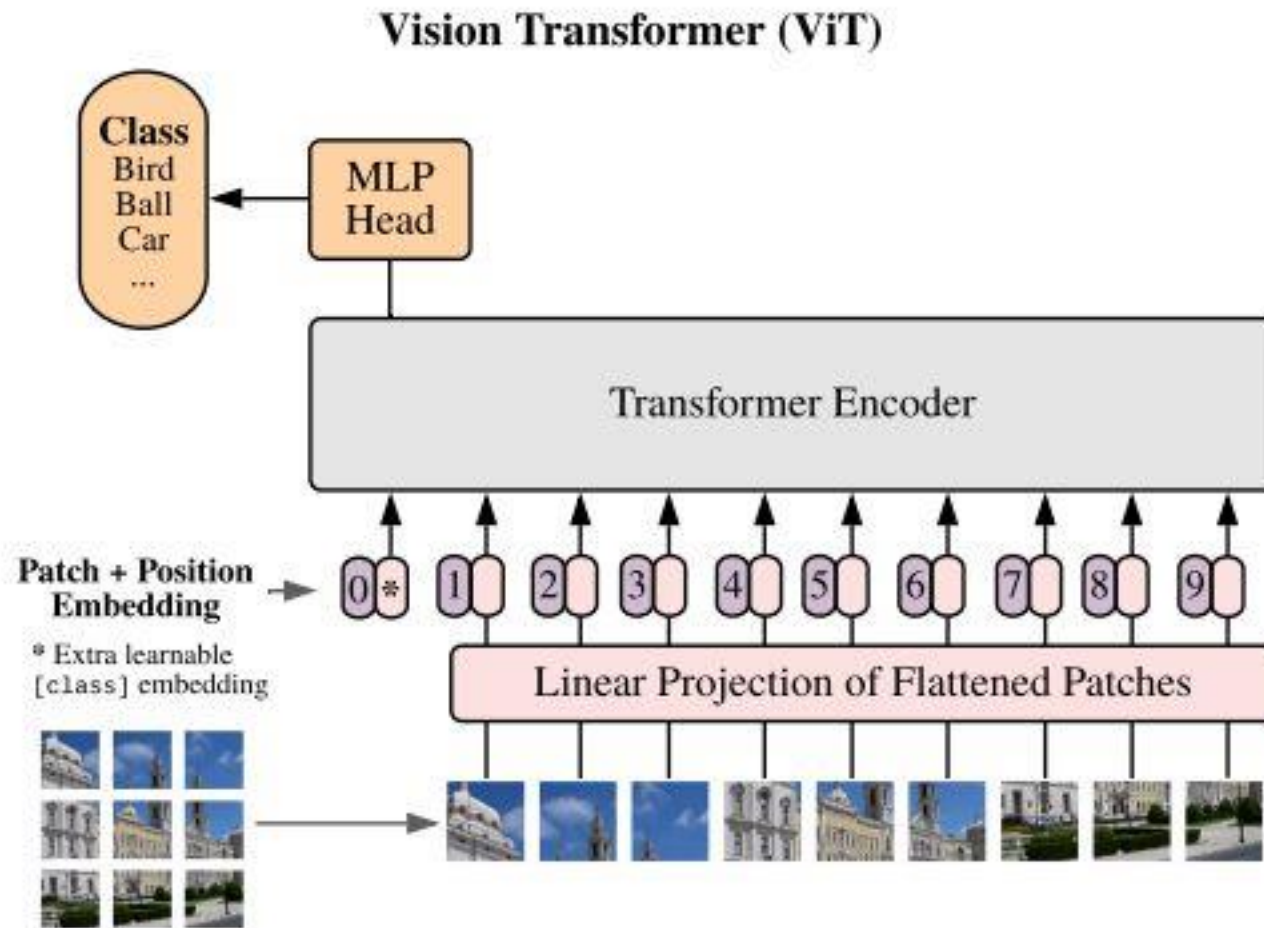
Sample 1

- Original Text: "bad acting ."
- Token IDs: [4, 11, 12, 0, 0, ..., 0] (*32 elements total*)
- Sentiment Label: Negative
- Actual Length: 3 tokens / Max Length: 32

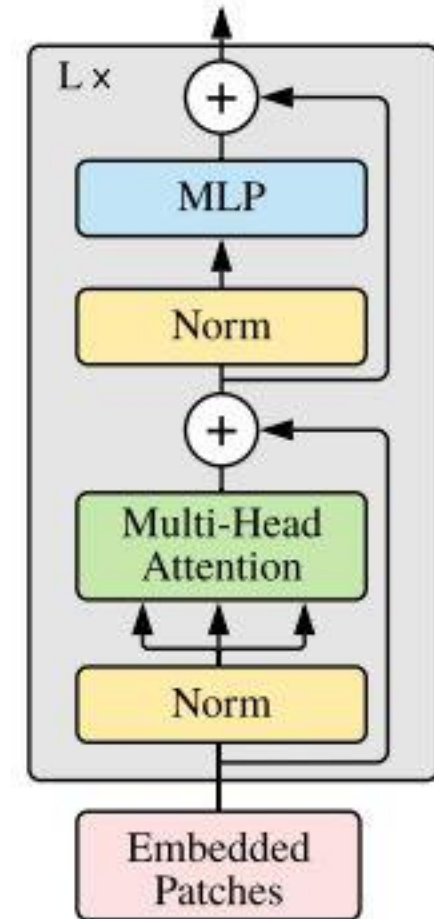
Sample 2

- Original Text: "good film !"
- Token IDs: [2, 7, 10, 0, 0, ..., 0] (*32 elements total*)
- Sentiment Label: Positive
- Actual Length: 3 tokens / Max Length: 32

Vision Transformer



Transformer Encoder



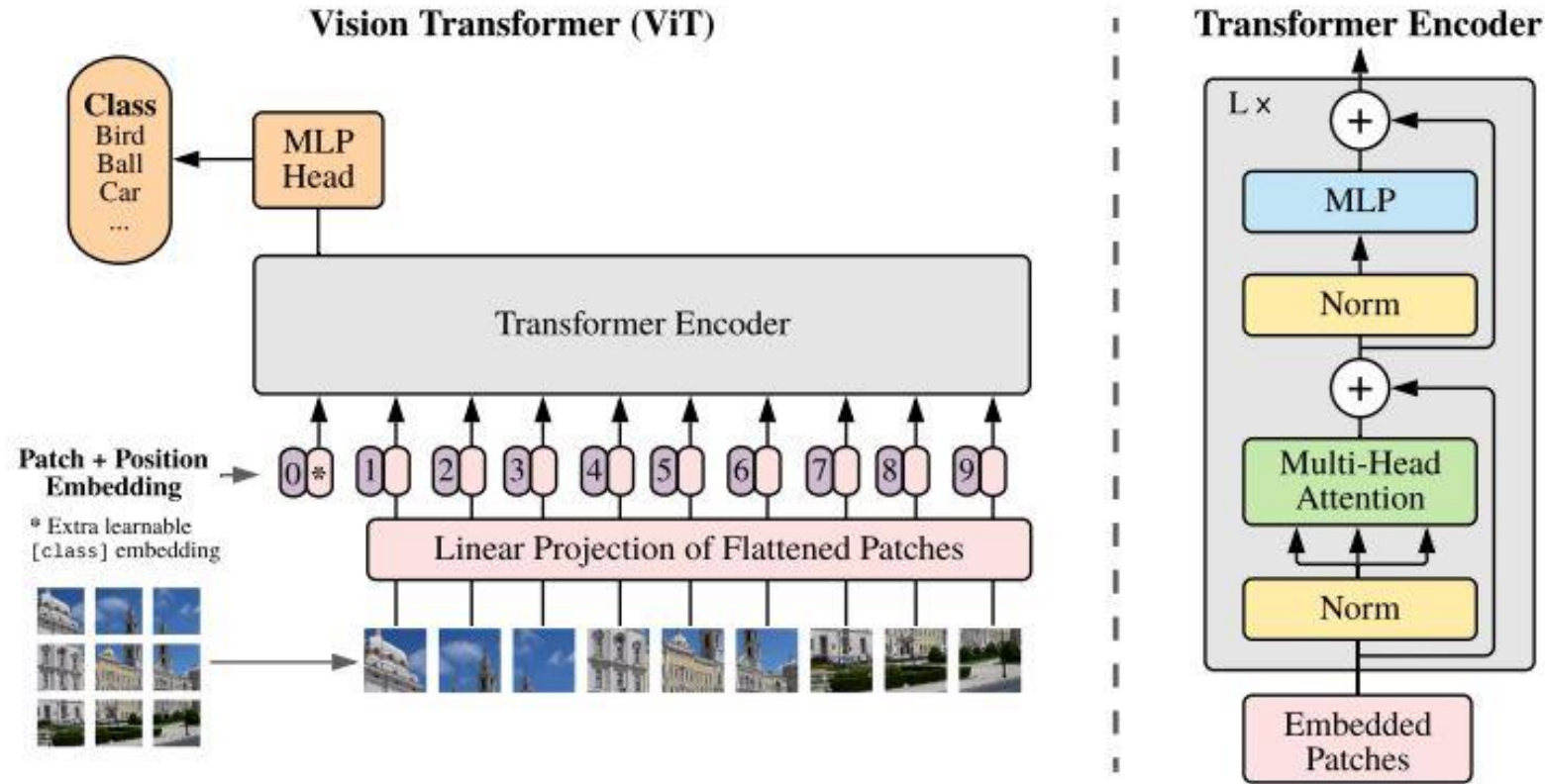
Powerful in Computer Vision

Vision Transformer

- 1. Patch-Based Approach:** ViT divides images into smaller patches instead of processing the whole image at once.
- 2. Embeddings:** Like in the original transformer for language, ViT uses embeddings to represent patch content and position. Position embeddings help capture spatial information.
- 3. Transformer Encoder:** ViT uses transformer encoders, including self-attention and feedforward layers, to capture relationships between patches on both global and local scales.
- 4. Self-Attention:** Self-attention lets the model weigh patch importance relative to others, capturing long-range dependencies and context effectively.

- 5. Hybrid Approach:** Current ViT combines CNNs and transformers. Patch embeddings often undergo convolutional processing before entering transformer layers. (*e.g.*, TransUNet)
- 6. Pretraining and Fine-Tuning:** ViT models are pretrained on a large dataset with a proxy task (*e.g.*, predicting image rotations) before fine-tuning for tasks like image classification.

Vision Transformer



(1) Patch Embedding

- **Input Processing:** Split image into fixed-size non-overlapping patches (e.g., 16×16 pixels), flatten each into a vector.
- *Example:* Input: 224×224 RGB image; Patch size: $16 \times 16 \rightarrow (224/16)^2 = 196$ patches; Each patch: $16 \times 16 \times 3 = 768$ -D vector.
- **Linear Projection:** Map patches to model dimension (e.g., 768-D) via learnable matrix.

(2) Position Embedding

- **Purpose:** Preserves spatial information (Transformers are order-agnostic).
- **Methods:** Learnable embeddings (ViT default); Sinusoidal functions (original Transformer).

Thank you!

Questions?